



SWING CONTRACT PRICING: WITH AND WITHOUT NEURAL NETWORKS

VINCENT LEMAIRE^{✉1}, GILLES PAGÈS^{✉1} AND CHRISTIAN YEO^{✉*1,2}

¹Sorbonne Université, Laboratoire de Probabilités Statistique et Modélisation,
UMR 8001, 75005 Paris, France

²Engie Global Markets, 1 place Samuel Champlain, 92400 Courbevoie, France

(Communicated by Johannes Muhle-Karbe)

ABSTRACT. We propose two parametric approaches to evaluate swing contracts with firm constraints. Our objective is to define approximations for the optimal control, which represents the amounts of energy purchased throughout the contract. The first approach involves approximating the optimal control by means of an explicit parametric function, where the parameters are determined using stochastic gradient descent based algorithms. The second approach builds on the first one, where we replace parameters in the first approach by the output of a neural network. Our numerical experiments demonstrate that by using Langevin based algorithms, both parameterizations provide, in a short computation time, better prices compared to state-of-the-art methods.

Introduction. With the energy market becoming increasingly deregulated, various derivative products have emerged, offering flexibility in delivery dates and purchased amounts of energy. *Swing* contracts, also known as *Take-or-Pay* contracts (see [47] for more details), are among the most widely traded contracts in gas and power markets. These contracts allow their holder to purchase amounts of energy at fixed exercise dates, subject to constraints. There exist two types of constraints: firm or penalized. In the firm constraints setting, the contract holder is not allowed to violate the constraints. In the penalized setting, violating the (global) constraints is penalized proportionally to the deficit/excess of consumption. Our paper focuses on the valuation of swing contracts with firm constraints.

The valuation of such swing contracts is more challenging than that of classic American-style contracts [13, 26, 35, 45], precisely because of the presence of constraints related to time and volume. From a probabilistic standpoint, the valuation of swing contracts appears as a Stochastic Optimal Control (*SOC*) problem. Here, the control represents the vector of volumes of energy to purchase at each exercise date. To solve this SOC problem, two groups of methods may be considered.

The first group concerns methods based on the so-called *Backward Dynamic Programming Principle* (BDPP). In this group, the price of the swing contract can be written as the solution of a dynamic programming equation (see [4, 3, 5, 27, 32, 47]). BDPP approach involves a conditional expectation also known as the “continuation

2020 *Mathematics Subject Classification.* Primary: 49M25, 93E20; Secondary: 93E35.

Key words and phrases. Swing contracts, stochastic control, stochastic approximation, neural network, Langevin dynamics, greeks.

*Corresponding author: Christian Yeo.

value” and its numerical computation is the main difficulty. The most commonly used method for computing the continuation value is the Longstaff and Schwartz one (see [35]) where it is approximated as successive orthogonal projection over subspaces spanned by a finite number of squared integrable random variables (see [5, 35]). Another method is based on optimal quantization (see [3, 41]) where the stochastic dynamics of the traded asset is approximated by space discretization of the whole process taking the shape of the so-called quantization tree. This tree is built offline by nearest neighbour projections of simulated paths of the underlying dynamics. One of the major problems encountered with BDPP approaches is the following. To compute the value of the contract at each exercise date, the maximisation is performed on geometric interval. In practice, this interval needs to be discretized leading to a loss of accuracy. Indeed, achieving a high level of accuracy in pricing the contract requires a finer discretization of the geometric interval, which in turn increases the computation time. Additionally, the Longstaff-Schwartz method faces a storage challenge since, to compute the continuation value, regression coefficients for each simulation and admissible cumulative consumption must be stored at each exercise date. This often leads to a memory overflow when a large number of simulations is required to obtain a price with a tight confidence interval. Meanwhile, the optimal quantization method suffers from the “curse of dimensionality” because it is well known that the rate of convergence of this method is of order $\mathcal{O}(N^{-\frac{1}{d}})$ (where N is the number of points of the quantization tree and d is the problem dimension).

An alternative approach to the BDPP one is to consider the valuation of swing contracts as a global optimization problem. As already mentioned, the valuation of swing contracts is equivalent to solve a *SOC* problem where the objective is to find a vector of purchase amounts that maximizes the expected value of cumulative cash flows up to the expiry of the contract. These cash flows have to be simulated leading to a stochastic optimization problem where the set of admissible controls is assimilated to a parameterized family of functions, the optimization being performed by a Stochastic Gradient Descent based on (Monte Carlo) simulations of the underlying dynamics. The use of parametric functions to approximate the optimal control reduces the control problem to a parametric optimization problem. The success of such approaches is highly depending on the choice of the parameterized family of functions which often requires a deep understanding of the optimal control behaviour. Note that, solving *SOC* problem with SGD based algorithms has already been considered in [2, 12, 23, 24, 36] for general *SOC* problems and more specially in [5] for swing contracts. In this paper, we introduce two parameterized families of functions to approximate the optimal control in the swing valuation problem and where one of the parameterizations is based on deep neural networks. To optimize both parameterizations, we consider two optimization algorithms based on stochastic gradient descent. The first optimization algorithm is the classic “Adaptive Moment Estimation” (Adam) algorithm which is widely used for stochastic approximation. As an alternative to Adam algorithm, we propose the use of the Preconditioned Stochastic Gradient Langevin Dynamics (PSGLD) which has demonstrated his effectiveness in recent studies for Bayesian learning. Our results show that using Langevin-type algorithms can accelerate the optimization of our parameterizations and leads to better prices compared to the state-of-the-art methods.

Our paper is organised as follows. Section 1. We describe swing contracts and recall the pricing framework. Section 2. We present our contribution by proposing two parameterized families of functions designed to approximate the optimal control. Section 3. We present generalities about stochastic approximation as well as two optimization algorithms (Adam and PSGLD) to optimize both parameterized controls. Finally in section 4, we present some additional results about the effectiveness of our methods.

1. On swing contracts. Due to their dual constraint (exercise dates and volume constraints) nature, swing contracts present some specificities when it comes to their evaluation. This section provides a reminder of the general background on swing contracts.

1.1. Description. A swing option is a commonly encountered derivative product on energy (gas and power) markets which allows its holder to buy amounts of energy q_ℓ at times $t_\ell = \frac{\ell T}{n}$, $\ell = 0, \dots, n-1$ (called exercise dates) until the contract maturity $t_n = T$. At each exercise date t_ℓ , the unitary purchase price (or strike price of the contract) is denoted K_ℓ . It can be constant (i.e. $K_\ell = K, \ell = 0, \dots, n-1$) or indexed on the price of either the underlying instrument or another one (oil for example). In this paper, we focus on the fixed strike price case but the case with indexed strike can be treated likewise.

One important feature of a swing option are the volume constraints. Swing option gives its holder a flexibility on the amount of energy he is allowed to purchase and this amount is subject to two kinds of (firm) constraints

- **Local constraints:** at each exercise date t_ℓ , the holder of the swing contract is allowed to buy at least $q_{\min}$ and at most $q_{\max}$ of the underlying i.e.,

$$0 \leq q_{\min} \leq q_\ell \leq q_{\max}, \quad 0 \leq \ell \leq n-1. \quad (1)$$

- **Global constraints:** the cumulative consumption up to the maturity of the contract must be not lower than $Q_{\min}$ and not greater than $Q_{\max}$ i.e.,

$$Q_n = \sum_{\ell=0}^{n-1} q_\ell \in [Q_{\min}, Q_{\max}], \quad \text{with } Q_0 = 0 \text{ and } 0 \leq Q_{\min} \leq Q_{\max} \leq +\infty. \quad (2)$$

In this paper, we only consider **firm constraints** which means that the holder of the contract is not allowed to violate the constraints. In this setting, the existence of an optimal (*bang-bang*) consumption had been established in [4]. A similar result is proved for the case with penalties in [5]. Recall that the penalty setting corresponds to the case where the holder of the swing contract is allowed to violate the global constraints but is faced to penalties proportional to the excess/deficit of consumption (purchased volumes of energy).

In the firm constraints case, the local and global constraints allow to draw the swing physical space which represents possible actions per exercise date.

1.2. Physical space. At each exercise date, due to firm constraints, the attainable cumulative volumes are upper and lower bounded by two functions $t_\ell \mapsto Q^{\text{down}}(t_\ell)$ and $t_\ell \mapsto Q^{\text{up}}(t_\ell)$

$$\begin{cases} Q^{\text{down}}(t_0) = 0, \\ Q^{\text{down}}(t_\ell) = \max(0, Q_{\min} - (n - \ell) \cdot q_{\max}), \quad \ell \in \{1, \dots, n-1\}, \\ Q^{\text{down}}(t_n) = Q_{\min}, \end{cases} \quad (3)$$

and

$$\begin{cases} Q^{up}(t_0) = 0, \\ Q^{up}(t_\ell) = \min(\ell \cdot q_{\max}, Q_{\max}), \ell \in \{1, \dots, n-1\}, \\ Q^{up}(t_n) = Q_{\max}. \end{cases} \quad (4)$$

These boundaries lead to the physical space of the swing contract, as drawn in Figure 1: they represent at each exercise date, the range of attainable cumulative consumptions.

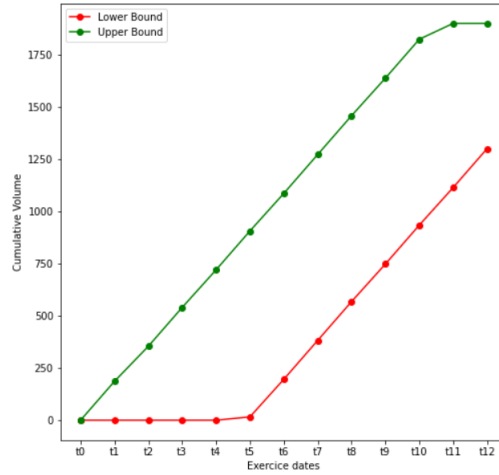


FIGURE 1. Illustration of the swing physical space with $q_{\min} = 0, q_{\max} = 180, Q_{\min} = 1300, Q_{\max} = 1900$ and $n = 12$ exercise dates.

In Figure 1, we assumed that $q_{\min} = 0$. This assumption can be made without loss of generality because, as shown in Appendix A (proof reproduced for the reader convenience), the general case can always reduce to the case where $q_{\min} = 0$ and $q_{\max} = 1$. Besides, if at an exercise date t_ℓ , we have bought amounts $q_0, \dots, q_{\ell-1}$

leading to a cumulative consumption $Q_\ell = \sum_{i=0}^{\ell-1} q_i$, then due to local constraints and the cumulative consumption boundaries, the actual range of attainable cumulative consumption at time $t_{\ell+1}$ is given by

$$\left[\underbrace{\max(Q^{down}(t_{\ell+1}), Q_\ell + q_{\min})}_{:=L_{\ell+1}(Q_\ell)}, \underbrace{\min(Q^{up}(\ell+1), Q_\ell + q_{\max})}_{:=U_{\ell+1}(Q_\ell)} \right]. \quad (5)$$

Thus, starting with a cumulative consumption Q_ℓ , at time t_ℓ , the range of admissible volumes (to be purchased) is

$$\left[\underbrace{L_{\ell+1}(Q_\ell) - Q_\ell}_{:=A_\ell^-(Q_\ell)}, \underbrace{U_{\ell+1}(Q_\ell) - Q_\ell}_{:=A_\ell^+(Q_\ell)} \right]. \quad (6)$$

It is important to note the *bang-bang* feature of the swing contracts in the firm constraints setting. It had been established in [4] that when volume constraints are integers and $Q_{\max} - Q_{\min}$ is a multiple of $q_{\max} - q_{\min}$, then the optimal consumption

(volume to purchase) is bang-bang. The latter means that, at each exercise date t_ℓ , the optimal consumption q_ℓ lies within $\{A_\ell^-, A_\ell^+\}$ (see (6)). The bang-bang feature is particularly valuable since it significantly reduces the computation time.

With these basic blocks, we can now set up the theoretical framework for the pricing of swing contracts.

1.3. Pricing and sensitivity calculus. The price at time t of the forward contract delivered at the maturity T is denoted by $F_{t,T}$. It is worth noting that, in the firm constraints setting aforementioned, there is no action to be done at the maturity T whereas in the case with penalty (see [5]), a penalty is applied, at the maturity T , in case of violation of global constraints defined in (2). This entails that, in practice and in the firm constraints setting, the underlying asset $F_{t,T}$ needs to be diffused until t_{n-1} unlike the case with penalty where we need to diffuse until $t_n = T$. We keep the notation t_{n-1} to denote the last exercise date following previous works [4, 3, 5].

In this paper (as in [3, 5]), we consider contracts on the spot price ($S_t = F_{t,t}$) even if in practice the spot turns out not to be a tradable instrument on market. In practice, the most encountered gas contract is the day-ahead forward contract whose price is $F_{t,t+1}$. But this case can be treated in the same way. We also consider a probability space $(\Omega, \mathcal{F}, \{\mathcal{F}_{t_\ell}, 0 \leq \ell \leq n-1\}, \mathbb{P})$ where $(\mathcal{F}_{t_\ell})_{0 \leq \ell \leq n-1}$ is the natural completed filtration of the process $(S_{t_\ell})_{0 \leq \ell \leq n-1}$.

The decision process $(q_\ell)_{0 \leq \ell \leq n-1}$ is defined on the same probability space and is supposed $\mathcal{F}_{t_\ell}$ -adapted. At each exercise date t_ℓ , by buying a volume q_ℓ , the holder of the contract makes an algebraic gross profit (or loss)

$$\psi_\ell(q_\ell, S_{t_\ell}) := q_\ell \cdot (S_{t_\ell} - K). \quad (7)$$

Given a cumulative consumption $Q \in \mathbb{R}_+$, an admissible strategy at time t_k is a vector $(q_k, \dots, q_{n-1})$ lying in the following set

$$\mathcal{A}_{k,Q}^{Q_{\min}, Q_{\max}} = \left\{ (q_\ell)_{k \leq \ell \leq n-1}, q_\ell : (\Omega, \mathcal{F}_{t_\ell}, \mathbb{P}) \mapsto [q_{\min}, q_{\max}], \right. \\ \left. \sum_{\ell=k}^{n-1} q_\ell \in [(Q_{\min} - Q)_+, Q_{\max} - Q] \right\}.$$

Using (6), this set also reads,

$$\mathcal{A}_{k,Q}^{Q_{\min}, Q_{\max}} = \left\{ (q_\ell)_{k \leq \ell \leq n-1}, q_\ell : (\Omega, \mathcal{F}_{t_\ell}, \mathbb{P}) \mapsto [A_\ell^-(Q_\ell), A_\ell^+(Q_\ell)], \right. \\ \left. Q_\ell = Q + \sum_{i=k}^{\ell-1} q_i \right\}, \quad (8)$$

with the convention $\sum_{i=k}^{k-1} q_i = 0$. Then for every non negative $\mathcal{F}_{t_{k-1}}$ -measurable random variable Q , the price of the swing contract at time t_k starting from a cumulative consumption Q is given by

$$P_k(S_{t_k}, Q) = \mathbb{P} - \operatorname{ess\,sup}_{(q_\ell)_{k \leq \ell \leq n-1} \in \mathcal{A}_{k,Q}^{Q_{\min}, Q_{\max}}} \mathbb{E} \left(\sum_{\ell=k}^{n-1} e^{-r_\ell(t_\ell - t_k)} \psi_\ell(q_\ell, S_{t_\ell}) | \mathcal{F}_{t_k} \right), \quad (9)$$

where the expectation is taken under the risk-neutral probability $\mathbb{P}$ and $(r_\ell)_\ell$ are interest rates that we will assume to be zero from now on. The price of the swing contract, at time $t_0 = 0$, is given by (S_{t_0} is assumed to be deterministic)

$$P_0 := \sup_{(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{Q_{\min}, Q_{\max}}} \mathcal{J}(q_0, \dots, q_{n-1}), \quad (10)$$

where, given an admissible strategy $(q_0, \dots, q_{n-1})$, the reward function $\mathcal{J}$ is defined as the expected value of cumulative future cash flows up to the expiry

$$\mathcal{J}(q_0, \dots, q_{n-1}) := \mathbb{E} \left(\sum_{\ell=0}^{n-1} \psi_\ell(q_\ell, S_{t_\ell}) \right). \quad (11)$$

The problem (10) appears to be a constrained stochastic control problem in which the aim is to find an admissible strategy (consumption) that maximizes the reward function $\mathcal{J}$. To achieve this, we propose two parametric strategies. Unless otherwise stated, for numerical computations, we use a one-factor model like in [5, 27]:

$$\frac{dF_{t,T}}{F_{t,T}} = \sigma_F \cdot e^{-\lambda(T-t)} dW_t, \quad t \leq T, \quad (12)$$

where $(W_t)_{t \geq 0}$ is a standard Brownian motion. As mentioned before, we deal with the spot price whose price can be derived by a straightforward application of Itô

$$S_t = F_{0,t} \cdot \exp \left(\sigma_F \cdot X_t - \frac{1}{2} \Sigma_t^2 \right), \quad (13)$$

where,

$$X_t = \int_0^t e^{-\lambda(t-s)} dW_s \quad \text{and} \quad \Sigma_t^2 = \frac{\sigma_F^2}{2\lambda} (1 - e^{-2\lambda t}).$$

Along with this model, we consider two settings (presented below). In both cases, we set $\lambda = 4$, $\sigma_F = 0.7$.

Case 1

$$n = 31 \text{ exercise dates} \quad q_{\min} = 0 \quad q_{\max} = 6 \quad Q_{\min} = 140 \quad Q_{\max} = 200.$$

Case 2 (as in [3, 5])

$$n = 365 \text{ exercise dates} \quad q_{\min} = 0 \quad q_{\max} = 6 \quad Q_{\min} = 1300 \quad Q_{\max} = 1900.$$

The computer used for all computations and simulations has the following characteristics: *Processor: Intel(R) Core(TM) i7-1185G7 @ 3.00GHz, 32Go of RAM, Microsoft Windows 10 Enterprise.*

Besides, for practitioners, prices of derivative products are closely linked to their sensitivities with respect to market data. These sensitivities are essential for hedging purposes, as they indicate how the price of a derivative product changes with the market. Computing sensitivities involves calculating derivatives of functions, and in our case, we need to differentiate a price with respect to some parameters of the underlying diffusion, where the price is given by (10). To achieve this, we rely on the so-called *envelope theorem*. Let us briefly recall a short background of this theorem.

Let $f(x, \alpha)$ and $g_j(x, \alpha)$, $j = 1, 2, \dots, m$ be real-valued continuously differentiable functions on $\mathbb{R}^{n+\ell}$, where $x \in \mathbb{R}^n$ are some variables and $\alpha \in \mathbb{R}^\ell$ are parameters,

and consider the constrained optimization problem

$$\begin{cases} \max_x f(x, \alpha) \\ \text{subject to } g_j(x, \alpha) \geq 0, \quad 1 \leq j \leq m. \end{cases}$$

We introduce the Lagrangian function,

$$\mathcal{L}(x, \mu, \alpha) = f(x, \alpha) + \langle \mu, g(x, \alpha) \rangle,$$

where $\mu \in \mathbb{R}^m$ is the Lagrange multipliers, $\langle \cdot, \cdot \rangle$ is the Euclidean inner-product and $g = (g_1, \dots, g_m)^\top$ (for a vector x , $x^\top$ denotes its transpose). Then we define the value function $V(\alpha) = f(x^*(\alpha), \alpha)$ where $x^*(\alpha)$ is a solution that maximizes the function $f(\cdot, \alpha)$. The following theorem gives the derivative of the value function V in case it is differentiable.

Theorem 1.1 (Envelope theorem). *Assume that V and $\mathcal{L}$ are continuously differentiable where $\frac{\partial \mathcal{L}}{\partial \alpha_k} = \frac{\partial f}{\partial \alpha_k} + \langle \mu, \frac{\partial g}{\partial \alpha_k} \rangle$. Then,*

$$\frac{\partial V(\alpha)}{\partial \alpha_k} = \frac{\partial \mathcal{L}(x^*(\alpha), \mu^*(\alpha), \alpha)}{\partial \alpha_k}, \quad k = 1, \dots, \ell.$$

In model (12), we deduce the following proposition which is a corollary of Theorem 1.1.

Proposition 1.2. *Let $(q_k^*)_{0 \leq k \leq n-1}$ be a solution of the problem (10). Assume that the function*

$$(F_{0,t_k})_{0 \leq k \leq n-1} \mapsto \mathbb{E} \left(\sum_{k=0}^{n-1} q_k^* \cdot \left(F_{0,t_k} \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2} \Sigma_{t_k}^2} - K \right) \right)$$

is continuously differentiable. Let P_0 be the price of the swing contract (10). Note that P_0 is a function of $(S_{t_k})_{0 \leq k \leq n-1}$ which in turns depends on $(F_{0,t_k})_{0 \leq k \leq n-1}$ in the model (12). Then for all $k = 0, \dots, n-1$, the delta (sensitivity of swing price with respect to the initial forward price) is given by

$$\frac{\partial P_0}{\partial F_{0,t_k}} = \mathbb{E} \left(q_k^* \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2} \Sigma_{t_k}^2} \right),$$

where X_{t_k} is defined in (13).

Proof. We define the following functions:

$$f((q_k)_{0 \leq k \leq n-1}, (F_{0,t_k})_{0 \leq k \leq n-1}) := \mathbb{E} \left(\sum_{k=0}^{n-1} q_k \cdot \left(F_{0,t_k} \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2} \Sigma_{t_k}^2} - K \right) \right),$$

$$g_1((q_k)_{0 \leq k \leq n-1}, (F_{0,t_k})_{0 \leq k \leq n-1}) := \sum_{k=0}^{n-1} q_k - Q_{\min},$$

$$g_2((q_k)_{0 \leq k \leq n-1}, (F_{0,t_k})_{0 \leq k \leq n-1}) := Q_{\max} - \sum_{k=0}^{n-1} q_k,$$

and for all $k = 0, \dots, n-1$:

$$g_{2k+3}((q_k)_{0 \leq k \leq n-1}, (F_{0,t_k})_{0 \leq k \leq n-1}) := q_k - q_{\min},$$

$$g_{2k+4}((q_k)_{0 \leq k \leq n-1}, (F_{0,t_k})_{0 \leq k \leq n-1}) := q_{\max} - q_k.$$

Then it suffices to prove that functions $f, g_1, g_2, \dots, g_{2n+2}$ are continuously differentiable in order to use the envelope theorem. Notice that this holds for functions

$g_1, g_2, \dots, g_{2n+2}$. It remains to prove it for function f . The latter reduces to show that function f is continuously differentiable in each of its components. For any $k = 0, \dots, n-1$, the random variable $q_k \cdot (F_{0,t_k} \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2}\Sigma_{t_k}^2} - K)$ is integrable since

$$\left| q_k \cdot (F_{0,t_k} \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2}\Sigma_{t_k}^2} - K) \right| \leq q_{\max} \cdot (F_{0,t_k} \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2}\Sigma_{t_k}^2} + K) \in \mathbb{L}_{\mathbb{R}}^1(\mathbb{P}).$$

Moreover, the function $F_{0,t_k} \mapsto q_k \cdot (F_{0,t_k} \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2}\Sigma_{t_k}^2} - K)$ is differentiable and its derivative does not depend on F_{0,t_k} . Furthermore,

$$\begin{aligned} \left| \frac{\partial}{\partial F_{0,t_k}} \left(q_k \cdot (F_{0,t_k} \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2}\Sigma_{t_k}^2} - K) \right) \right| &= \left| q_k \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2}\Sigma_{t_k}^2} \right| \\ &\leq q_{\max} \cdot e^{\sigma_F \cdot X_{t_k}} \in \mathbb{L}_{\mathbb{R}}^1(\mathbb{P}). \end{aligned}$$

Then thanks to Lebesgue theorem to interchange derivation and integral, the function f is continuously differentiable in all F_{0,t_k} . Likewise one may also show that for all $k = 0, \dots, n-1$ the function f is continuously differentiable in q_k , and one may use the envelope theorem. For any $k = 0, \dots, n-1$ we introduce the Lagrangian of the problem (10)

$$\begin{aligned} \mathcal{L} \left((q_k)_{0 \leq k \leq n-1}, \mu, (F_{0,t_k})_{0 \leq k \leq n-1} \right) &= \mathbb{E} \left(\sum_{k=0}^{n-1} q_k \cdot (F_{0,t_k} \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2}\Sigma_{t_k}^2} - K) \right) \\ &\quad + \langle \mu, g((q_k)_{0 \leq k \leq n-1}, (F_{0,t_k})_{0 \leq k \leq n-1}) \rangle, \end{aligned}$$

where $g = (g_1, \dots, g_{2n+2})^\top$ and $\mu \in \mathbb{R}^{2n+2}$. Then it follows from envelope theorem that for any $k = 0, \dots, n-1$:

$$\begin{aligned} \frac{\partial P_0}{\partial F_{0,t_k}} &= \mathbb{E} \left(\sum_{k=0}^{n-1} q_k^* \cdot \frac{\partial}{\partial F_{0,t_k}} \left(F_{0,t_k} \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2}\Sigma_{t_k}^2} - K \right) \right) \\ &= \mathbb{E} \left(\sum_{k=0}^{n-1} q_k^* \cdot e^{\sigma_F \cdot X_{t_k} - \frac{1}{2}\Sigma_{t_k}^2} \right). \end{aligned}$$

This completes the proof. $\square$

We have outlined the theoretical framework for pricing swing contracts. To obtain practical numerical solutions, we introduce some parametric methods based on a global optimization approach.

2. Swing pricing: A global optimization approach. In this paper, we propose to solve the optimization problem (10) through a global optimization approach. It is worth noting that the use of that kind of approaches in the swing pricing context is not new and has demonstrated its advantages when compared to dynamic programming based methods like the classic method of Longstaff and Schwartz (see [35]). We refer the reader to the paper of Barrera-Esteve et al. [5] where a global optimization approach had been used to evaluate a swing contract in case with penalties.

The strategy is *bang-bang* so at each exercise date t_k , given a cumulative consumption Q_k , the random variable q_k lies in a 2-state set $\{A_k^-(Q_k), A_k^+(Q_k)\}$. Since this random variable q_k is measurable with respect to $\mathcal{F}_{t_k}$, there exists $B_k \in \mathcal{F}_{t_k}$ such that

$$q_k = A_k^-(Q_k) + (A_k^+(Q_k) - A_k^-(Q_k)) \mathbf{1}_{B_k}, \quad 0 \leq k \leq n-1.$$

Our approach involves approximating the decision function $\mathbf{1}_{B_k}$ with a parametric function that takes values in the range $[0, 1]$. More precisely, we use at each time step t_k a parametric function

$$\begin{aligned} \chi_k : \mathbb{R}^d \times \mathbb{R}_+ \times \Theta &\rightarrow \mathbb{R}, \\ (S, Q, \theta) &\mapsto \chi_k(S, Q, \theta), \end{aligned} \quad (14)$$

where Θ is a finite-dimensional parameter space that will be specified later. We compose this parametric function with the logistic function σ to map $\mathbb{R}$ to the interval $[0, 1]$. The parametric strategy writes then (for all $0 \leq k \leq n-1$)

$$q_k(S_k, Q_k^\theta, \theta) = A_k^-(Q_k^\theta) + (A_k^+(Q_k^\theta) - A_k^-(Q_k^\theta))\sigma(\chi_k(S_{t_k}, Q_k^\theta, \theta)), \quad (15)$$

where the cumulative consumption Q_k^θ is updated as follows

$$Q_0^\theta = 0 \quad \text{and} \quad Q_k^\theta = \sum_{\ell=0}^{k-1} q_\ell(S_{t_\ell}, Q_\ell^\theta, \theta) \quad \text{for } 1 \leq k \leq n-1.$$

Using this parametric strategy, the optimization problem (10) using the global revenue (11) is approximated by the following finite-dimensional optimization problem

$$\theta^* \in \operatorname{argmax}_{\theta \in \Theta} \bar{\mathcal{J}}(\theta) \quad \text{with} \quad \bar{\mathcal{J}}(\theta) = \mathbb{E} \left(\sum_{k=0}^{n-1} \psi_k(q_k(S_{t_k}, Q_k^\theta, \theta), S_{t_k}) \right). \quad (16)$$

The function $\bar{\mathcal{J}}$, which corresponds to the reward function following a parametric strategy, is typically unknown, and we will employ stochastic optimization algorithms to maximize it. This initial optimization step will sometimes be referred to as a learning or training phase later on. We will detail the algorithms used in Section 3.

After completing the learning of the parameter θ^* , we implement a Monte Carlo estimator using the simulation of new scenarios, independent of those used in the stochastic optimization procedure. We then simulate M_e paths $(S_{t_k}^{[m]})_{0 \leq k \leq n-1}$ (for $1 \leq m \leq M_e$) and the swing contract price is given by

$$\hat{P}_0 = \frac{1}{M_e} \sum_{m=1}^{M_e} \sum_{k=0}^{n-1} \psi_k(q_k(S_{t_k}^{[m]}, Q_k^{\theta^*}, \theta^*), S_{t_k}^{[m]}), \quad (17)$$

with the associated confidence interval. We refer to this second step as the evaluation phase.

2.1. Explicit payoff-volume parameterization (PV strat). A first choice of parametric function χ_k introduced in (14) is defined as follows. Let $\Theta = \mathcal{M}(n \times 3)$ be the set of all n -by-3 real matrices. Given $\theta \in \Theta$, for $0 \leq k \leq n-1$ we denote by $\theta_k = (\theta_{k1}, \theta_{k2}, \theta_{k3}) \in \mathbb{R}^3$ the $k+1$ -th row of matrix θ . At each timestep t_k , the parametric function χ_k is defined by

$$\chi_k(S, Q, \theta) = \theta_{k1}(S - K) + \theta_{k2}\eta(Q) + \theta_{k3}, \quad \text{with} \quad \eta(Q) = \frac{Q - Q_{\min}}{Q_{\max} - Q_{\min}}. \quad (18)$$

To condense this notation, we use the dot product in $\mathbb{R}^3$ and write $\chi_k(S, Q, \theta) = \langle \theta_k, I \rangle$ with $I = (S - K, \eta(Q), 1)$. The function η computes the normalized remaining purchasing capacity. The choice of the function η is discussed in Remarks 2.1 and 2.2. In case $Q_{\min} = Q_{\max}$, one may use $\eta(Q) = \frac{Q - Q_{\min}}{Q_{\min}}$.

The idea behind the representation (2.5) is the following. We believe that the optimal decision/control depends on two key features: the payoff $S - K$ (short-term feature) and, the remaining purchasing capacity (long-term feature) due to global constraints. This remaining purchasing capacity is modelled by function η which importance is discussed in Remark 2.1. We opt for this linear representation because in this setting, coefficients θ_{k1}, θ_{k2} carry a straightforward interpretation, denoting the relative significance of the corresponding feature (either the payoff $S - K$ or the remaining purchasing capacity $\eta(Q)$) in the explanation of optimal control. For instance, at an exercise date t_k , if the volume Q satisfies the global constraints, then the optimal purchasing decision is mostly dependent on the payoff, meaning θ_{k1} needs to be high enough. Indeed, in this case, the optimal decision is to purchase the maximum possible volume in case the payoff is positive i.e. $S > K$ and the minimum possible volume otherwise. Conversely, if the global constraints are not met at that date, even with a negative payoff, one might be compelled to purchase the maximum allowed volume to adhere to the global constraints, emphasizing the importance of the feature $\eta(Q)$ to the detriment of the payoff. In summary, representation (2.5) provides a simple yet intuitive way to capture the heuristic described above. Besides, it allows to reduce the search space by imposing a particular shape that we believe to be optimal (refer to the previous section for details). Later on, we will see that this simple parametric function (18) is flexible enough to yield good numerical results.

Remark 2.1 (Importance of function η). At first glance, the function η may be seen as a simple normalization of the cumulative consumption. However, it should be noted that several other normalizations were tested and did not lead to a good strategy. We tested: $\eta(Q) = Q, \frac{Q - Q_{\min}}{q_{\max} - q_{\min}}$ and some normalizations depending on the step k , for example $\frac{k \cdot q_{\max} - Q}{q_{\max} - q_{\min}}$ and $\frac{Q - Q^{\text{down}}(t_{k+1})}{Q^{\text{up}}(t_{k+1}) - Q^{\text{down}}(t_{k+1})}$.

Remark 2.2 (Remaining capacity *Versus* Cumulative Consumption). We observed that the remaining purchasing capacity is a more crucial factor than the current cumulative consumption when determining the volume to buy at each exercise date. This contrasts with the neural network approach tested in [5], which uses the current cumulative consumption. We have noticed that when using $\eta(Q) = Q$, our algorithm tended to purchase the maximum possible volume at the contract's outset, as long as the payoff was positive. This behaviour suggests that the algorithm did not take into account the fact that purchasing capacity diminishes as the contract approaches its expiration, due to global constraints.

Remark 2.3 (Complexity). Using this parameterization, it is worth noting that the finite-dimensional optimization problem (16) in the training phase corresponds to a space of dimensionality $3n$.

2.2. Neural network parameterization (NN strat). One way to improve the previous parameterization is to replace each parameter $\theta_k \in \mathbb{R}^3$ (in *PV strat*) with the output of one artificial neural network that depend on some information available at time t_k . More precisely, we use a feedforward neural network (FNN) denoted by Φ and defined by the composition of affine functions and activation functions. Let us be more precise. Consider a network depth of $I \geq 1$ and $\underline{d} := \min(d_1, \dots, d_I)$, where for $i = 1, \dots, I$, the positive integer d_i denotes the number of neurons in the i^{th} layer. The FNN Φ , defined from $\mathbb{R}^d$ to $\mathbb{R}^p$ (in our case $p = 3$), is written as follows

$$x \in \mathbb{R}^d \mapsto \Phi_{\underline{d}}(x; \theta) = a_I^{\theta_I} \circ \phi \circ a_{I-1}^{\theta_{I-1}} \circ \dots \circ \phi \circ a_1^{\theta_1}(x) \in \mathbb{R}^p, \quad (19)$$

where

- $\theta := (\theta_1, \dots, \theta_I) \in \Theta := \prod_{i=1}^I (\mathcal{M}(d_i \times d_{i-1}) \times \mathbb{R}^{d_i})$ with $d_0 = d$ and $d_I = p$.
- for each layer $i = 1, \dots, I$ the affine function $a_i^{\theta_i} : \mathbb{R}^{d_{i-1}} \rightarrow \mathbb{R}^{d_i}$ is defined by $a_i^{\theta_i}(x) = W_i x + b_i$ where $\theta_i = (W_i, b_i) \in \mathcal{M}(d_i \times d_{i-1}) \times \mathbb{R}^{d_i}$.
- $\phi : \mathbb{R} \rightarrow \mathbb{R}$ is an activation function, which is applied component-wise to a given vector. In the numerical experiments, we only use ReLU (Rectified Linear Unit) function i.e. $x \mapsto \max(x, 0)$.

FNNs of form (19) have a powerful approximation capacity which is stated by the Universal Approximation Theorem [14, 25]. Among others, a version of the latter theorem (from [31]) is reproduced below (see Theorems 2.4 and 2.5) for the reader convenience. Besides, some quantitative error bounds have been studied in the literature [1, 7, 6] with a quite general review in [18]. To state the universal approximation theorem, let us consider the following class of FNNs of form (19) with depth I and activation function ϕ ,

$$\mathcal{NN}_{I,d} := \{\Phi_d(\cdot; \theta) : \mathbb{R}^d \rightarrow \mathbb{R}, \theta \in \Theta\} \quad (20)$$

and denote by $\mathcal{NN}_{I,\infty} := \bigcup_{d \geq 1} \mathcal{NN}_{I,d}$. Then, we have the following theorems giving,

at least theoretically, what feedforward neural networks can learn.

Theorem 2.4. *Assume that the activation function ϕ is non-constant and bounded. Let μ denote a probability measure on $\mathbb{R}^d$, then for any $I \geq 2$, $\mathcal{NN}_{I,\infty}$ is dense in the space $\mathbb{L}_\mu^2(\mathbb{R}^d)$ of squared μ -integrable functions from $\mathbb{R}^d$ to $\mathbb{R}$.*

Theorem 2.5. *Assume that the activation function ϕ is a non constant, bounded and continuous function, then, when $I = 2$, $\mathcal{NN}_{I,\infty}$ is dense into the space of continuous functions from $\mathbb{R}^d$ to $\mathbb{R}$ for the topology of the uniform convergence on compact sets.*

These theorems state that FNNs of form (19) with one hidden layer can approximate a wide class of functions for a given accuracy, whence their characterization as *universal approximators*.

In this second approach based on neural networks, we replace each $\mathbb{R}^3$ -valued vector $\theta_k = (\theta_{k1}, \theta_{k2}, \theta_{k3})$ of (18) by the output of a $\mathbb{R}^3$ -valued feedforward neural network $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^3$ of form (19). More precisely, we consider

$$\chi_k(S, Q, \theta) = \langle \Phi(t_k, S - K, \eta(Q); \theta), I \rangle, \quad \text{with } I = (S - K, \eta(Q), 1), \quad (21)$$

and $\eta(Q) = \frac{Q - Q_{\min}}{Q_{\max} - Q_{\min}}$. We represent in Figure 2 an illustration of the architecture of the neural network.

The concept driving this modelling approach remains centered on modelling the contributions of specific features, as in (18). However, in this iteration, these contributions are modelled as output of a neural network. This approach enables the integration of non-linearity in the modelling of these contributions, contrasting with (18), thus enlarging the spectrum of functions to explore. Besides, it is worth noting that this modelling into two steps is mainly due to the fact that we have a priori knowledge of the optimal control's shape: the optimal strategy is bang-bang and we use a sigmoid function as a smooth approximation. Without such prior knowledge, it would be more relevant to model the optimal control using a real-valued neural network. However, in our specific setting where we have a priori knowledge of the optimal control's shape, it turns out that our modelling is more effective than

directly modelling the optimal control as the output of a real-valued neural network. The latter means that, given a small number of iterations (1000 iterations in our experiments), our method achieves a significantly better maximum (i.e. higher prices). This is probably because, learning the optimal control from scratch is likely to take more time than starting from a particular shape based on prior knowledge.

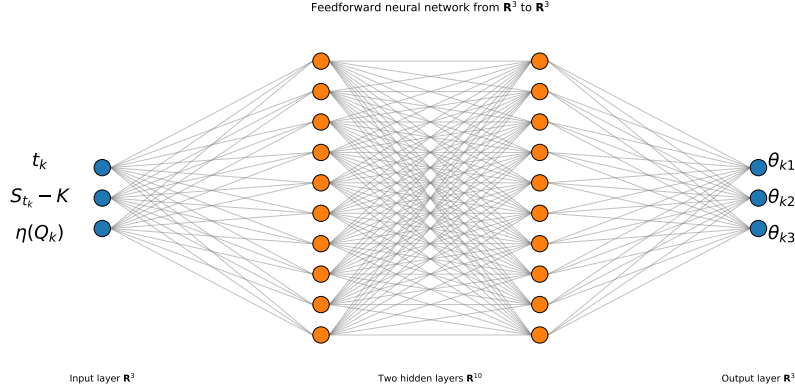


FIGURE 2. Illustration of feedforward neural network architecture.

Remark 2.6 (Complexity). With this parameterization, the parameter space involved in training phase (see (16)) has a dimension equal to $\sum_{i=1}^I d_i d_{i-1} + d_i$ which does not depend on the number of exercise dates n , unlike the preceding parameterization PV strat where the parameter space dimension increases linearly with the number of exercise dates (as pointed out in Remark 2.3).

3. Stochastic optimization for learning phase. In this section, we will review some recent and efficient stochastic optimization algorithms for solving the high-dimensional problem (16). To establish our notation, let's start by revisiting stochastic gradient ascent for maximizing the deterministic function $\bar{J}$, which can be expressed as an expectation

$$\bar{J}(\theta) = \mathbb{E}[J(\theta, Z)] \quad \text{with} \quad J(\theta, Z) = \sum_{k=0}^{n-1} \psi_k(q_k(S_{t_k}, Q_k^\theta, \theta), S_{t_k})$$

and the notation $Z := (S_{t_0}, \dots, S_{t_{n-1}})$. Adhering to the standard notations of stochastic approximation (refer to [29, 44]), let us define $H : \Theta \times \mathbb{R}^n$ as the gradient of $-J$ with respect to θ . We aim to find a root of $h(\theta) = \mathbb{E}[H(\theta, Z)]$. In practice, we employ automatic differentiation to compute the local gradient H

$$H(\theta, Z) = -\nabla_\theta J(\theta, Z) = -\sum_{k=0}^{n-1} \nabla_\theta \psi_k(q_k(S_{t_k}, Q_k^\theta, \theta), S_{t_k}) \quad (22)$$

with $Z := (S_{t_0}, \dots, S_{t_{n-1}})$. Then, the standard stochastic gradient descent to approximate θ^* a zero of h writes then

$$\theta_{k+1} = \theta_k - \gamma_{k+1} H(\theta_k, Z^{[k+1]}), \quad (23)$$

with $(Z^{[k]})_{k \geq 1}$ an *i.i.d.* sequence of copies of $Z = (S_{t_0}, \dots, S_{t_{n-1}})$ and $(\gamma_n)_{n \geq 0}$ a real sequence decreasing to 0 satisfying the two conditions

$$\sum_{n \geq 0} \gamma_n = +\infty \quad \text{and} \quad \sum_{n \geq 0} \gamma_n^2 < +\infty. \quad (24)$$

The sequence $(\gamma_n)_{n \in \mathbb{N}}$ is called the algorithm step or learning rate and has to be chosen carefully. It is often taken decreasing by plateau. Practitioners often use the mini-batch version of (23). That is, the local gradient in (23) computed over a single simulation is replaced with an average over a small number B of simulations i.e.,

$$\theta_{k+1} = \theta_k - \frac{\gamma_{k+1}}{B} \sum_{b=1}^B H(\theta_k, Z^{[k+1],b}), \quad (25)$$

with $(Z^{[k],b})_{k \geq 1, 1 \leq b \leq B}$ an *i.i.d.* sequence of copies of $Z = (S_{t_0}, \dots, S_{t_{n-1}})$.

The convergence of SGD had been widely studied in the literature. For Lipschitz continuous convex objective functions, some convergence results may be found in [39, 49]. When both smoothness and strong convexity of the objective function hold, a convergence analysis is provided in [37]. We refer the reader to [21, 33] for the non-convex setting and to [46] for non-smooth case.

Several different variants of the standard procedure (23) have been developed. In this paper, we use and compare two of them namely, the Adaptive Moment Estimation (Adam) algorithm and the Preconditioned Stochastic Gradient Langevin Dynamics (PSGLD) algorithm.

Remark 3.1. Hereafter, we will keep the learning rate constant over the training phase i.e. $\gamma_k = \gamma$ as often set by practitioners.

3.1. Adaptive moment estimation (Adam). Adam [28] is an efficient stochastic optimization method that only requires first-order gradients with little memory requirement. The method computes individual adaptive learning rates for different parameters from estimates of first gradient moments. Practically, this is done by means of a preconditioning matrix P (see updating (26)). Adam can be seen as a combination of RMSprop¹ and Stochastic Gradient Descent with momentum [43]. It replaces the gradient in the stochastic procedure (23) with the (scaled) moving average of past gradients values until the considered iteration. The procedure is the following:

$$\theta_{n+1} = \theta_n - \gamma P_{n+1} \cdot \widehat{M}_{n+1}, \quad (26)$$

where $\widehat{M}_{n+1}$ is a scaled moving average of local gradients H (22) and $(P_n)_n$ is a sequence of preconditioning (diagonal) matrix where diagonal entries are inverse of the scaled moving average of squares of gradients H until iteration n (see Algorithm 1). In Algorithm 1 (and also in Algorithm 2), we adopt the following notations. Given a gradient $g = (g_1, \dots, g_d) \in \mathbb{R}^d$, the square component-by-component of g denoted $g \odot g$ is defined by:

$$g \odot g = (g_1^2, \dots, g_d^2). \quad (27)$$

¹Unpublished optimization algorithm designed for neural networks, first proposed by Geoffrey Hinton in a Coursera course https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

In the same spirit, for some matrix $A = (a_{i,j})_{1 \leq i,j \leq d}$, $B = (b_{i,j})_{1 \leq i,j \leq d}$ where $b_{i,j} \neq 0$ for all $1 \leq i,j \leq d$, we define a component-by-component division as follows,

$$A \oslash B = (c_{i,j})_{1 \leq i,j \leq d} \quad \text{where } c_{i,j} = a_{i,j}/b_{i,j}. \quad (28)$$

Algorithm 1: Adam updating. λ is a small correction term to avoid division by zero. Id_d denotes the $d \times d$ identity matrix.

Input :

- Step size γ .
- $\mu_1, \mu_2 \in [0, 1)$: exponential decay rates for the moment estimates.
- Initialize parameter vector θ_0 .

Output: Optimal parameter θ^* .

```

1 /* Initialize the moving average for the gradient and its
   square. */
2  $M_0 = 0$  (first moment vector),  $MS_0 = 0$  (second moment vector).
3 while  $\theta_n$  not converged do
4      $\hat{g}_{n+1} \leftarrow H(\theta_n, Z^{[n+1]})$ . /* Estimate of gradient using (22) and
       automatic differentiation. */
5      $M_{n+1} \leftarrow \mu_1 \cdot M_n + (1 - \mu_1) \cdot \hat{g}_{n+1}$ . /* Update biased first moment
       estimate. */
6      $MS_{n+1} \leftarrow \mu_2 \cdot MS_n + (1 - \mu_2) \cdot \hat{g}_{n+1} \odot \hat{g}_{n+1}$ . /* Update biased second
       raw moment estimate. */
7      $\hat{M}_{n+1} \leftarrow M_n / (1 - \mu_1^{n+1})$ . /* Compute bias-corrected first moment
       estimate. */
8      $\hat{MS}_{n+1} \leftarrow MS_n / (1 - \mu_2^{n+1})$ . /* Compute bias-corrected second raw
       moment estimate. */
9      $P_{n+1} \leftarrow \text{diag} \left( Id_d \oslash (\lambda \cdot Id_d + \sqrt{\hat{MS}_{n+1}}) \right)$ . /* Compute
       preconditioning matrix. */
10     $\theta_{n+1} \leftarrow \theta_n - \gamma P_{n+1} \cdot \hat{M}_{n+1}$ . /* Update parameters using (23) or
       (25). */
11 end while
12 return  $\theta_n$ .
```

It is worth noting that, in case of mini-batches, a loop over all mini-batches has to be added in Algorithm 1 and inside the *while* loop. Adam uses the squared gradients to scale the learning rate like RMSprop and it takes advantage of momentum by using the moving average (driven by μ_1, μ_2) of the gradient instead of the gradient itself like SGD with momentum. The scaling of the learning rate by the squared gradient allows to solve gradient magnitude problem. Otherwise, sometimes the gradients may be huge and sometimes small, which intricate the choice of a learning rate. The exponential moving average of the gradient allows to “denoise” the estimation of the gradient.

Adam has become the default algorithm for stochastic optimization and especially when it comes to train a neural network. Its theoretical convergence had been studied in the literature by its authors [28] in the convex setting and by other authors in the non-convex setting [17, 50]. In this paper, we do not limit ourselves to Adam updating. We propose to explore another algorithm based on Langevin dynamics and which shares, as Adam, good properties when it comes to escape traps. By traps we mean an unstable or repelling critical points of the loss function to be minimized.

3.2. Preconditioned Stochastic Gradient Langevin Dynamics (PSGLD).

Stochastic Gradient Langevin Dynamics (SGLD) has been introduced by Gelfand and Mitter in [22] to minimize a potential J and more recently brought back to light for Bayesian learning [48] to approximate a posterior distribution given some data. The paradigm of SGLD is to inject in Stochastic Gradient Descent an exogenous (Gaussian) noise in additive way at each update of the procedure, so that it makes it converge toward an invariant distribution almost concentrated on argmin J or, in its simulated annealing version, to converge in probability toward argmin J . The use of Langevin based optimization algorithms is relatively new in the context of stochastic control problems and had demonstrated its effectiveness in practice [12]. We refer the reader to Appendix E for theoretical details on the link between Langevin dynamics and optimization problems.

We keep the same notation as in Section 3. Recall that we aim to maximize the function $\bar{J}(\theta) = \mathbb{E}[J(\theta, Z)]$ and we consider $H : \Theta \times \mathbb{R}^n$ as the gradient of $-J$ with respect to θ (see (22)). After preliminary numerical tests (following see [10]), we finally implemented the preconditioned Langevin procedure from the RMSprop procedure (see [34]) as follows,

$$\theta_{n+1} = \theta_n - \gamma P_{n+1} \cdot H(\theta_n, Z^{[n+1]}) + \sigma \sqrt{\gamma} P_{n+1} \cdot \eta_{n+1}, \quad (29)$$

where,

- ▷ $(Z^{[n]})_n$ is a sequence of *i.i.d.* copies of $Z = (S_{t_0}, \dots, S_{t_{n-1}})$.
- ▷ $(\eta_n)_n$ is a sequence of *i.i.d.* copies of $\eta \sim \mathcal{N}(0, I_d)$.
- ▷ $(P_n)_n$ is a sequence of diagonal $d \times d$ matrix called *preconditioners* (as in Adam (26)) and which entries depend on a moving average of the gradient past values.

The pseudo-code of the preconditioned stochastic gradient langevin dynamic is presented in Algorithm 2, where we consider a constant step setting i.e. γ and σ . From a practical point of view the injection of noise helps the algorithm to escape from traps (see [38]). Besides the convergence of Langevin algorithm has been widely studied. One may refer to [19, 20] when the sequence $(\sigma_n)_n$ is constant i.e. $\sigma_n = \sigma$ and to [11, 40] when the sequence is decreasing.

3.3. Adam versus PSGLD. We implement *PV strat* and *NN strat* with both Adam and PSGLD optimization algorithms and in the *case 1* setting. This has been done using *PyTorch* toolbox and with *Nvidia A100-PCIE 40GB* as GPU device. The code can be found using this link: https://github.com/ChristianYeo/swing_parametric_strategies. As a benchmark, we have implemented the well-known method of Longstaff and Schwartz with canonical polynomial functions of degree 3 yielding a price equal to 65.14 ($IC_{95\%} : [65.08, 65.21]$). The average computation time for the Longstaff-Schwartz method is about 4 minutes when using 10^6 simulations.

Algorithm 2: PSGLD updating. λ is a small correction term to avoid division by zero. Id_d denotes the $d \times d$ identity matrix.

Input :

- Step size γ .
- $\beta \in [0, 1)$: exponential decay rate for the moment squared estimates
- Initialize parameter vector θ_0 .

Output: Optimal parameter θ^* .

```

1 /* Initialize the square moving average of the gradient.      */
2  $MS_0 = 0$  (second moment vector).
3 while  $\theta_n$  not converged do
4   /* Compute local gradient using (22) and automatic
      differentiation                                          */
5    $\hat{g}_{n+1} \leftarrow H(\theta_n, Z^{[n+1]})$  /* Update biased second raw moment
      estimate                                                  */
6    $MS_{n+1} = \beta \cdot MS_n + (1 - \beta) \cdot g_{n+1} \odot g_{n+1}$  /* Compute preconditioner
      (diagonal) matrix                                        */
7    $P_{n+1} \leftarrow \text{diag} \left( Id_d \odot (\lambda \cdot Id_d + \sqrt{MS_{n+1}}) \right)$ 
8   /* Update parameters                                       */
9    $\theta_{n+1} \leftarrow \theta_n - \gamma P_{n+1} \cdot g_{n+1} + \sigma \sqrt{\gamma} \cdot \mathcal{N}(0, (P_{n+1} P_{n+1}^\top))$ 
10 end while
11 return  $\theta_n$ .
```

In our experiments, we noticed that with *case 1* (even with *case 2*) setting, regardless the chosen method, the Monte Carlo estimator of the swing price (17) exhibits a high variance. Thus, for the sake of robust comparison, we adopt the following protocol. For all tables in this section, swing prices are computed by performing 100 times training phase then evaluation phase (as explained in Section 2) and the 100 resulting price estimators (17) are averaged. For each of these 100 replications, the training phase is performed using one batch of size $B = 2^{14}$, a learning rate $\gamma = 0.1$ and in the evaluation phase, we use $M_e = 10^6$ simulations.

In what follows, we denote by N the number of iterations in the stochastic procedure (25), by $\widehat{P}_0$ the (raw) price resulting from *PV strat* or *NN strat* and given by (17). $\widehat{P}_0^{BG}$ denotes the price given by forcing the consumption rule (either (18) for *PV strat* or (21) for *NN strat*) to be strictly bang-bang. That is, we shrink the logistic function σ in (15) i.e.,

$$\bar{\sigma}(\chi) := \mathbf{1}_{\{\chi \geq 0\}} \in \{0, 1\}, \quad (30)$$

where χ denotes the decision function (either (18) or (21)). Results are recorded in Tables 1 and 2 for *PV strat* and in Tables 2 and 4 for *NN strat*.

N	$\widehat{P}_0$	$\widehat{P}_0^{BG}$	Time (s)
3000	65.11 ([65.04, 65.18])	65.15 ([65.08, 65.22])	68.4
1000	65.02 ([64.95, 65.08])	65.18 ([65.12, 65.24])	21.3

TABLE 1. Results for *PV strat* using Adam optimization algorithm. A confidence interval (95%) is in brackets. Column “Time” includes both training and valuation times.

σ	β	$\widehat{P}_0$	$\widehat{P}_0^{BG}$	Time (s)
$1 \cdot e^{-6}$	0.8	65.19 ([65.12, 65.26])	65.21 ([65.13, 65.28])	21.3
$1 \cdot e^{-6}$	0.9	65.21 ([65.15, 65.28])	65.23 ([65.17, 65.29])	21.3
$1 \cdot e^{-5}$	0.9	65.22 ([65.16, 65.28])	65.23 ([65.18, 65.29])	21.6

TABLE 2. Results for *PV strat* using PSGLD optimization algorithm. A confidence interval (95%) is in brackets. Column “Time” includes both training and valuation times. We used $N = 1000$ iterations and $\lambda = 1 \cdot e^{-10}$.

Tables 1 and 2 suggest that Adam requires several iterations and, as a result, increases computation time when using a learning rate of 0.1. Whereas, with only $N = 1000$ iterations, using PSGLD optimization algorithm yields a better price. This fact is also observable in *NN strat* (see Tables 3 and 4). It seems that adding some noise in the optimization procedure helps to converge quickly.

N	$\widehat{P}_0$	$\widehat{P}_0^{BG}$	Time (s)
3000	65.22 ([65.16, 65.28])	65.23 ([65.17, 65.29])	150.4
1000	65.13 ([65.05, 65.20])	65.22 ([65.15, 65.29])	53.2

TABLE 3. Results for *NN strat* using Adam optimization algorithm. For the neural network architecture, we used 2 hidden layers ($I = 2$) with 10 units per layer ($q_1 = 10, q_2 = 10$). Values in brackets are confidence intervals (95%). Columns “Time” includes the training and the valuation times.

Following the performance of PSGLD updating, we consider this algorithm in the remainder along with the following configuration: $\sigma = 1 \cdot e^{-6}, \beta = 0.8, \lambda = 1 \cdot e^{-10}$. In the latter setting, one may compute sensitivity of the swing price with respect to the initial forward price (see Figure 3).

Note that for both optimization algorithms (Adam and PSGLD), different other hyperparameters have been tested and results are reported in Appendix D.

3.4. Practitioner’s corner: Transfer learning. We observed that, when the swing contract has several exercise dates or when we use more than one batch per iteration, the execution time increases drastically (see Appendix D). To illustrate

σ	β	$\widehat{P}_0$	$\widehat{P}_0^{BG}$	Time (s)
$1 \cdot e^{-6}$	0.9	65.23 ([65.16, 65.30])	65.21 ([65.14, 65.28])	53.4
$1 \cdot e^{-5}$	0.9	65.23 ([65.14, 65.31])	65.22 ([65.13, 65.30])	52.3
$1 \cdot e^{-6}$	0.8	65.27 ([65.20, 65.35])	65.26 ([65.18, 65.33])	51.3

TABLE 4. Results for *NN strat* using PSGLD optimization algorithm. For the neural network architecture, we used 2 hidden layers ($I = 2$) with 10 units per layer ($q_1 = 10, q_2 = 10$). Values in brackets are confidence intervals (95%). Columns “Time” includes the training and the valuation times. We used $N = 1000$ iterations and $\lambda = 1 \cdot e^{-10}$.

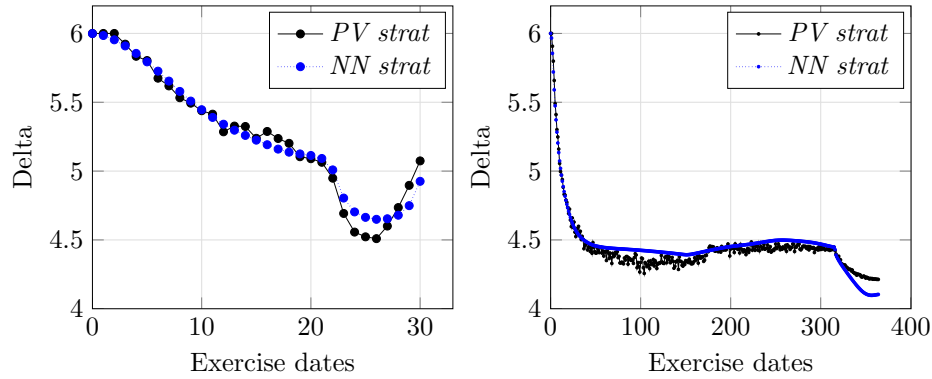


FIGURE 3. Delta forward in the settings of case 1 (left) and case 2 (right) using *PV strat* and *NN strat*.

this point, let us consider a swing contract with an expiry in one year and a daily exercise right (thus 365 exercise dates). Alongside this contract, we consider the diffusion model (12) for the underlying asset with the setting of *case 2*. The results are reported in Table 5. Notice that, as observed in [5], the Longstaff-Schwartz method generates numerical instabilities when evaluating this contract at a daily granularity. Moreover, it is worth noting that, we get better prices than optimal quantization method like in [3].

For all tables in this section, we used one batch of size $B = 2^{14}$, $M_e = 10^8$, $\gamma = 0.1$ and $N = 1000$ iterations.

	$\widehat{P}_0$	$\widehat{P}_0^{BG}$	(T_{train}, T_e)
<i>PV strat</i>	2690.10 ([2689.20, 2691.00])	2692.59 ([2691.68, 2693.49])	(274.1, 74.7)
<i>NN strat</i>	2693.74 ([2692.83, 2694.64])	2694.12 ([2693.21, 2695.02])	(680.9, 398.3)

TABLE 5. Results for a one-year swing contract. A confidence interval (95%) is in brackets. T_{train} denotes the training time and T_e the valuation time.

From Table 5 one may notice that the computation time is not negligible even if, compared to other methods (like Longstaff-Schwartz) it remains reasonable. To reduce it, we propose a method based on the so-called *transfer learning* (for details see [42]). The latter refers to a machine learning method where a model developed for a task is reused as a starting point for a model on a second and similar task. This method may accelerate the training phase when the swing contract has several exercise dates. In the latter case, we proceed as follows. We first consider an aggregated version. That is, we consider a swing contract with less exercise dates and where the local constraints are aggregated. For instance for a swing contract over one year with daily exercise right and with local constraints $q_{\min} = 0, q_{\max} = 6$, we may consider a contract with one exercise per month (the middle of each month) and where for example, for months with 30 days, the local constraints become $q_{\min} = 0 \times 30 = 0, q_{\max} = 30 \times 6 = 180$. Then we perform the training phase, as explained in Section 2, with a few number of iterations. The resulting parameters are used as an initial guess for the actual pricing problem (with several exercise dates). Note that in the *PV strat* the number of parameters depends on the number of exercise dates. Thus, to initialize the parameters in the problem with 365 exercise dates with the parameters obtained in the aggregated problem (with 12 exercise dates), we assume that all days within the same month behave the same. Therefore the initial guess parameters for all days within the same month are those obtained in the aggregated problem for this month. It turns out that this allows to reach more quickly a descent optimum.

	$\widehat{P}_0$ $\widehat{P}_0^{BG}$	T_{agg}	(T_{train}, T_e)
PV strat	2688.73 ([2687.82, 2689.63]) 2692.81 ([2691.90, 2693.71])	4.8	(83.2, 76.3)
NN strat	2693.22 ([2692.31, 2694.13]) 2694.12 ([2693.21, 2695.02])	6.9	(206.5, 398.1)

TABLE 6. Results for a one-year swing contract using transfer learning. We used 500 iterations for the aggregated contract and 300 iterations for the actual one-year contract. T_{agg} denotes the training time for the aggregated contract, T_{train} and T_e denote respectively the training time and the valuation time for the actual contract.

We can notice from Table 6 that by means of transfer learning, the training time is reduced by a factor of 3 without degrading the accuracy. Besides, always with in mind the aim of reducing the computation time, transfer learning can be used differently. In a few words, by means of transfer learning, when the market data change or when the contracts settings change, we do not need to start all over again. To illustrate this point, let us consider below three cases: **M1**, **M2**, **M3**, representing some shifts of market/contract settings. Recall that the baseline settings are those considered above (*case 2* setting).

In Table 7, we consider three different market move scenarios described as follows. In the first case, we bump the initial forward curve from 20 to 22. In the second

<i>Case M1</i>	<i>Case M2</i>	<i>Case M3</i>
$F_{0,t_k} = 22$	$(Q_{\min}, Q_{\max}) = (1400, 2000)$	$K = 18$

TABLE 7. Market data move scenarios

case, we change the global constraints from $Q_{\min} = 1300$ and $Q_{\max} = 1900$ to $Q_{\min} = 1400$ and $Q_{\max} = 2000$. In the final case, we reduce the strike price from 20 to 18. We could have changed the local constraints but as already pointed out, swing pricing can always reduce to the case where $q_{\min} = 0, q_{\max} = 1$ (see Appendix A). Now the question is the following: Could our baseline model give accurate prices without training again ? Or could it help us to get accurate prices quickly ? The answers to these questions are recorded in the following tables.

Cases	Reuse	Retrain	(T_{train}, T_e)
Case M1	5942.65 ([5941.64, 5943.65]) 5944.27 ([5943.26, 5945.27])	6005.75 ([6004.70, 6006.79]) 6008.54 ([6007.50, 6009.58])	(85.4, 77.2)
Case M2	2514.86 ([2513.90, 2515.81]) 2517.26 ([2516.30, 2518.21])	2516.99 ([2516.04, 2517.93]) 2518.73 ([2517.78, 2519.67])	(85.0, 76.9)
Case M3	5655.98 ([5655.06, 5656.89]) 5657.67 ([5656.75, 5658.58])	5744.91 ([5743.95, 5745.86]) 5747.39 ([5746.43, 5748.34])	(85.9, 77.05)

TABLE 8. Results for *PV strat.* Column “Reuse” provides results when the baseline model parameters are reused as is. Column “Retrain” gives results when we retrain with only 300 iterations with the baseline model parameters used as starting values. T_{train} denotes the training time and T_e the valuation time. The testing computation time is the same when we reuse the model as when we retrain.

Tables 8 and 9 demonstrate that in the case where global constraints change (*M2*), using the baseline parameters without additional training gives prices that are comparable to those obtained by training with a few iterations starting from the baseline parameters. Furthermore, it is important to note that increasing the number of iterations beyond 300 (we tested up to 1000 iterations in our experiments) does not lead to better prices than those obtained with 300 iterations (see columns labelled “Retrain”). This suggests that reusing the baseline parameters can speed up convergence. This observation remains true regardless of the case (*M1*, *M2* or *M3*).

4. Numerical experiments. We now perform additional simulations to demonstrate the effectiveness of the two parameterizations proposed in this paper.

4.1. Three factor model. We first consider a three factor model whose dynamics is given by:

$$\frac{dF_{t,T}}{F_{t,T}} = \sigma_{F,1} \cdot e^{-\lambda_1(T-t)} dW_t^1 + \sigma_{F,2} \cdot e^{-\lambda_2(T-t)} dW_t^2 + \sigma_{F,3} \cdot e^{-\lambda_3(T-t)} dW_t^3, \quad (31)$$

Cases	Reuse	Retrain	(T_{train}, T_e)
Case M1	5981.17 ([5980.14, 5982.19])	6011.08 ([6010.03, 6012.12])	(203.1, 397.8)
	5982.57 ([5981.54, 5983.59])	6011.62 ([6010.57, 6012.66])	
Case M2	2509.97 ([2509.01, 2510.90])	2516.92 ([2515.96, 2517.87])	(203.7, 397.5)
	2511.08 ([2510.12, 2512.03])	2517.58 ([2516.62, 2518.53])	
Case M3	5713.80 ([5712.85, 5714.72])	5749.97 ([5749.01, 5750.92])	(203.4, 397.6)
	5715.24 ([5714.34, 5716.17])	5750.51 ([5749.55, 5751.46])	

TABLE 9. Results for *NN strat*. Column “Reuse” provides results when the baseline model parameters are reused as is. Column “Retrain” gives results when we retrain with only 300 iterations with the baseline model parameters used as starting values. T_{train} denotes the training time and T_e the valuation time. The testing computation time is the same whether we reuse the model or we retrain it.

where for all $1 \leq i, j \leq 3$, the instantaneous correlation is given by:

$$\langle dW^i, dW^j \rangle_t = \begin{cases} dt & \text{if } i = j \\ \rho_{i,j} \cdot dt & \text{if } i \neq j \end{cases}$$

In this model, the spot price is given by

$$S_t = F_{0,t} \cdot \exp \left(\langle \sigma_F, X_t \rangle - \frac{1}{2} \Sigma_t^2 \right),$$

where $\sigma_F = (\sigma_{F,1}, \sigma_{F,2}, \sigma_{F,3})^\top$, $X_t = (X_t^1, X_t^2, X_t^3)^\top$ and for all $1 \leq i \leq 3$,

$$X_t^i = \int_0^t e^{-\lambda_i(t-s)} dW_s^i \quad \text{and} \quad \Sigma_t^2 = \sum_{1 \leq i, j \leq 3} \rho_{i,j} \frac{\sigma_{F,i} \sigma_{F,j}}{\lambda_i + \lambda_j} (1 - e^{-(\lambda_i + \lambda_j)t}).$$

We set the following configuration: $\sigma_{F,i} = 0.7$, $\lambda_i = 1.5$, $\rho_{i,j} = \rho \cdot \mathbf{1}_{i \neq j} + \mathbf{1}_{i=j} \in [-1, 1]$ along with *case 1* settings. We use $N = 1000$ iterations, a learning rate $\gamma = 0.1$ and $M_e = 1 \cdot e^8$ simulations for the valuation. Results are recorded in Tables 10 and 11.

ρ	$\widehat{P}_0$	$\widehat{P}_0^{BG}$
0.6	172.71 ([172.51, 172.90])	172.72 ([172.52, 172.92])
0.3	147.79 ([147.63, 147.95])	147.78 ([147.62, 147.95])
-0.2	91.02 ([90.92, 91.12])	91.02 ([90.92, 91.12])

TABLE 10. Results using *PV strat*. A confidence interval (95%) is in brackets. For each result, the execution time (training plus valuation) is roughly equal to 22s.

ρ	$\widehat{P}_0$	$\widehat{P}_0^{BG}$
0.6	173.05 ([172.85, 173.24])	172.98 ([178.78, 173.17])
0.3	147.97 ([147.81, 148.14])	147.92 ([147.75, 148.08])
-0.2	91.18 ([91.08, 91.28])	91.13 ([91.03, 91.23])

TABLE 11. Results using *NN strat*. A confidence interval (95%) is in brackets. For the neural network architecture, we used $I = 2$ layers with $q_1 = q_2 = 10$ units. For each result, the execution time (training plus valuation) is roughly equal to 45s.

It should be noted that there exists another way to implement *NN strat* which takes into account the structure of forward prices. Specifically, in the three-factor framework (31), the forward price depends on state variables that are components of the $\mathbb{R}^3$ -valued random vector X_{t_ℓ} . In this context, one can include X_{t_ℓ} as an additional input to the neural network described in (21), in addition to time t_ℓ , payoff $S_{t_\ell} - K$, and volume normalization $M(Q_\ell)$. That is, $I_\ell = (t_\ell, S_{t_\ell} - K, X_{t_\ell}, M(Q_\ell))^\top \in \mathbb{R}^{d+3}$, where d is the dimension of X_{t_ℓ} (in the three-factor model, $d = 3$). This approach can help to capture the correlation structure in a multi-factor framework. The prices resulting from this alternative approach are presented in Table 12.

ρ	$\widehat{P}_0$	$\widehat{P}_0^{BG}$
0.6	172.76 ([172.56, 172.95])	172.70 ([172.50, 172.89])
0.3	147.95 ([147.78, 148.11])	147.91 ([147.75, 148.07])
-0.2	91.11 ([91.01, 91.20])	91.07 ([90.97, 91.17])

TABLE 12. Results using *NN strat* including state variables. A confidence interval (95%) is in brackets. For the neural network architecture we used $I = 2$ layers with $q_1 = q_2 = 10$ units. For each result, the execution time (training plus valuation) is roughly equal to 50s.

It appears that, regardless of whether the model is multi-factor or not, both strategies (*PV strat* and *NN strat*) are able to determine the optimal volume to purchase at each exercise date, based only on the payoff and the cumulative volume. In the following section, we will evaluate the performance of our strategies on a more complex diffusion model.

4.2. Multi-curve forward diffusion. We finally consider a multi-curve model. At a certain valuation date t , we consider p risk factors. Each risk factor $i \in \{1, \dots, p\}$, denoted by F_{t, T_i} , is a forward contract observed at date t and expiring at date T_i . It is modelled through an instantaneous volatility function $\sigma_t(T_i)$ with a dynamics given by,

$$\frac{dF_{t, T_i}}{F_{t, T_i}} = \sigma_t(T_i) dW_t(T_i), \quad (32)$$

where $(W_t(T_i))_{t \geq 0}$ are standard Brownian motions with instantaneous correlation given by:

$$\langle dW.(T_i), dW.(T_j) \rangle_t = \rho_{i,j} dt.$$

The model (32) implies to diffuse one curve per risk factor F_{t,T_i} defined by its maturity date T_i . Likewise, the spot at a given date generates its own risk factor. But due to high correlation (at any date, two risk factors are very correlated when maturities are very close), one can see that only a few risk factors really impact the price. In practice, forward contracts delivering in the same month are highly correlated. Thus we will consider that there are as many factors as months. For example, for a contract on a whole year $p = 12$; therefore 12 (correlated) Brownian motions overall. In this model, we consider a valuation date on March 17th, 2021 and a swing contract delivering on the period January 2022-December 2022. The swing volume constraints are: $q_{\min} = 0, q_{\max} = 1, Q_{\min} = 180, Q_{\max} = 270$. For the diffusion model, the correlation matrix is (*in percentage*):

$$\begin{pmatrix} 100 & 99.62 & 98.61 & 91.12 & 91.12 & 91.12 & 90.24 & 90.24 & 90.24 & 87.2 & 87.2 & 87.2 \\ 99.62 & 100 & 98.91 & 91.65 & 91.65 & 91.65 & 90.68 & 90.68 & 90.68 & 87.8 & 87.8 & 87.8 \\ 98.61 & 98.91 & 100 & 93.89 & 93.89 & 93.89 & 92.79 & 92.79 & 92.79 & 89.94 & 89.94 & 89.94 \\ 91.12 & 91.65 & 93.89 & 100 & 100 & 100 & 99.47 & 99.47 & 99.47 & 97.06 & 97.06 & 97.06 \\ 91.12 & 91.65 & 93.89 & 100 & 100 & 100 & 99.47 & 99.47 & 99.47 & 97.06 & 97.06 & 97.06 \\ 91.12 & 91.65 & 93.89 & 100 & 100 & 100 & 99.47 & 99.47 & 99.47 & 97.06 & 97.06 & 97.06 \\ 90.24 & 90.68 & 92.79 & 99.4 & 99.47 & 99.4 & 100 & 100 & 100 & 97.32 & 97.32 & 97.32 \\ 90.24 & 90.68 & 92.79 & 99.47 & 99.47 & 99.47 & 100 & 100 & 100 & 97.32 & 97.32 & 97.32 \\ 90.24 & 90.68 & 92.79 & 99.47 & 99.47 & 99.47 & 100 & 100 & 100 & 97.32 & 97.32 & 97.32 \\ 87.2 & 87.8 & 89.94 & 97.06 & 97.06 & 97.06 & 97.32 & 97.32 & 97.32 & 100 & 100 & 100 \\ 87.2 & 87.8 & 89.94 & 97.06 & 97.06 & 97.06 & 97.32 & 97.32 & 97.32 & 100 & 100 & 100 \\ 87.2 & 87.8 & 89.94 & 97.06 & 97.06 & 97.06 & 97.32 & 97.32 & 97.32 & 100 & 100 & 100 \end{pmatrix}$$

Volatilities and initial forward prices are assumed to be constant per month. For months in 2022, the square of the instantaneous volatility $\sigma_t^2(T_i)$ is given by Figure 4 and values are: 45.09%, 44.88%, 42.63%, 40.57%, 36.99%, 34.37%, 31.18%, 29.59%, 30.22%, 30.6%, 31.05%, 30.25%. The initial forward curve is given by Figure 5 and values are: 20.07, 20, 19.6, 17.4, 16.75, 16.5, 16.56, 16.53, 16.71, 17.31, 18.31, 18.64.

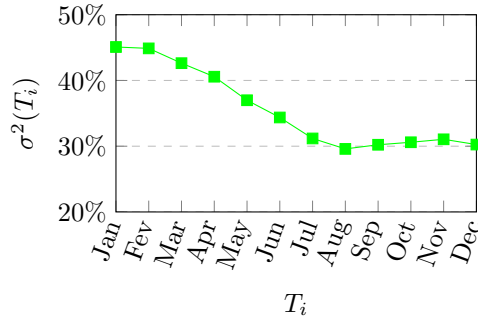


FIGURE 4. Square of instantaneous volatility per risk factor observed on 17-March-2021.

Finally, the strike price corresponds to the average of all initial forward prices; which gives 17.865. For this diffusion model, *PV strat* does not perform well. This is probably due to the multi-curve framework which is more complex than a

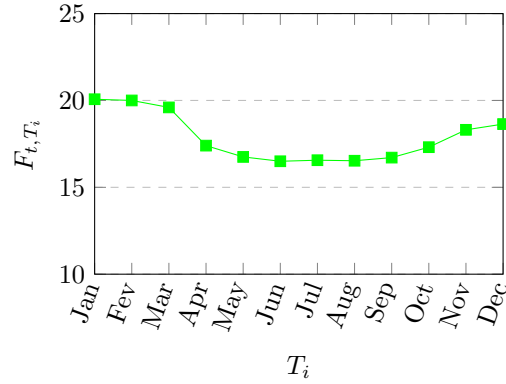


FIGURE 5. Forward prices per risk factor observed on 17-March-2021.

multifactor model. However, by modifying *NN strat* as in the preceding section by adding, in neural network inputs, Brownian motions $(W_{t_k}(T_i))_{1 \leq i \leq p}$ at each exercise date t_k , allows to achieve very good prices. We compare the latter implementation of *NN strat* with a method based on a static replication of swing contracts by means of spread options. The latter method aims at approximating swing options prices by selecting a combination of spread options (*SO*) where the underlying is driven by the same dynamics (32). The selection is done through a linear programming method. Results are recorded in Table 13.

$\widehat{P}_0$	$\widehat{P}_0^{BG}$	<i>SO</i>
708.46 ([705.76, 711.16])	711.32 ([708.61, 714.03])	645.3

TABLE 13. Results using *NN strat*. A confidence interval (95%) is in brackets. For the neural network architecture we used $I = 2$ layers with $q_1 = q_2 = 50$ units.

Conclusion. We introduced two parametric methods for the pricing of swing contracts based on a global optimization approach. The first method involves an explicit parameterization of the optimal control based on some heuristics, while the second method improves upon the first by using neural networks. We conducted numerical experiments to compare two optimization algorithms (Adam and PSGLD). Our results demonstrate that using Langevin based optimization algorithms, namely PSGLD, allows us to achieve better prices with short computation time. Additionally, we tested our neural network parameterization in various complex diffusion models and demonstrated its robustness and accuracy compared to state-of-the-art methods. We can hence conclude that our neural network based method is well-suited to the pricing swing options.

Acknowledgments. The third author is grateful for the financial support provided by Engie Global Markets via CIFRE agreement and would like to thank Asma Meziou and Frederic Muller for throughout insights on swing contracts.

Appendix A. Swing contract decomposition. We aim to prove that any swing contract can be reduced to a normalized contract, namely a swing contract with local constraints $q_{\min} = 0, q_{\max} = 1$. The swing contract price is given by

$$P_0 = \operatorname{ess\,sup}_{(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{Q_{\min}, Q_{\max}}} \mathbb{E} \left(\sum_{\ell=0}^{n-1} q_\ell \times (S_{t_\ell} - K) \right),$$

where

$$\mathcal{A}_{0,0}^{Q_{\min}, Q_{\max}} = \left\{ (q_\ell)_{0 \leq \ell \leq n-1}, q_\ell : (\Omega, \mathcal{F}_{t_\ell}^S, \mathbb{P}) \mapsto [q_{\min}, q_{\max}], \sum_{\ell=0}^{n-1} q_\ell \in [Q_{\min}, Q_{\max}] \right\}.$$

It follows from the linearity of the expectation that,

$$\begin{aligned} P_0 &= q_{\min} \times \mathbb{E} \left(\sum_{\ell=0}^{n-1} (S_{t_\ell} - K) \right) + (q_{\max} - q_{\min}) \\ &\quad \times \operatorname{ess\,sup}_{(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{Q_{\min}, Q_{\max}}} \mathbb{E} \left(\sum_{\ell=0}^{n-1} \frac{q_\ell - q_{\min}}{q_{\max} - q_{\min}} \times (S_{t_\ell} - K) \right). \end{aligned}$$

The first term in the last equality is given by

$$\mathbb{E} \left(\sum_{\ell=0}^{n-1} (S_{t_\ell} - K) \right) = \sum_{\ell=0}^{n-1} \mathbb{E} (S_{t_\ell}) - n \cdot K,$$

and can be easily computed using either closed formula (depending on the underlying diffusion model) or Monte-Carlo method. Let us consider the second term. Let $(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{Q_{\min}, Q_{\max}}$ and define $\tilde{q}_\ell = \frac{q_\ell - q_{\min}}{q_{\max} - q_{\min}}$. Note that $(\tilde{q}_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{\tilde{Q}_{\min}, \tilde{Q}_{\max}}$ where

$$\mathcal{A}_{0,0}^{\tilde{Q}_{\min}, \tilde{Q}_{\max}} = \left\{ (q_\ell)_{0 \leq \ell \leq n-1}, q_\ell : (\Omega, \mathcal{F}_{t_\ell}, \mathbb{P}) \mapsto [0, 1], \sum_{\ell=0}^{n-1} q_\ell \in [\tilde{Q}_{\min}, \tilde{Q}_{\max}] \right\},$$

and

$$\tilde{Q}_{\min} = \frac{(Q_{\min} - n \cdot q_{\min})_+}{q_{\max} - q_{\min}}, \quad \tilde{Q}_{\max} = \frac{(Q_{\max} - n \cdot q_{\min})_+}{q_{\max} - q_{\min}}.$$

Thus,

$$\begin{aligned} \mathbb{E} \left(\sum_{\ell=0}^{n-1} \frac{q_\ell - q_{\min}}{q_{\max} - q_{\min}} \times (S_{t_\ell} - K) \right) &= \mathbb{E} \left(\sum_{\ell=0}^{n-1} \tilde{q}_\ell \times (S_{t_\ell} - K) \right) \\ &\leq \operatorname{ess\,sup}_{(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{\tilde{Q}_{\min}, \tilde{Q}_{\max}}} \mathbb{E} \left(\sum_{\ell=0}^{n-1} q_\ell \times (S_{t_\ell} - K) \right). \end{aligned}$$

Therefore taking the supremum yields,

$$\begin{aligned} &\operatorname{ess\,sup}_{(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{Q_{\min}, Q_{\max}}} \mathbb{E} \left(\sum_{\ell=0}^{n-1} \frac{q_\ell - q_{\min}}{q_{\max} - q_{\min}} \times (S_{t_\ell} - K) \right) \\ &\leq \operatorname{ess\,sup}_{(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{\tilde{Q}_{\min}, \tilde{Q}_{\max}}} \mathbb{E} \left(\sum_{\ell=0}^{n-1} q_\ell \times (S_{t_\ell} - K) \right). \end{aligned}$$

Conversely let $(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{\tilde{Q}_{\min}, \tilde{Q}_{\max}}$ and define $\tilde{q}_\ell = q_{\min} + (q_{\max} - q_{\min}) \cdot q_\ell \in [q_{\min}, q_{\max}]$. It follows $\sum_{\ell=0}^{n-1} q_\ell \in [Q_{\min}, Q_{\max}]$ so that $(\tilde{q}_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{Q_{\min}, Q_{\max}}$. Thus,

$$\begin{aligned} \mathbb{E} \left(\sum_{\ell=0}^{n-1} q_\ell \times (S_{t_\ell} - K) \right) &= \mathbb{E} \left(\sum_{\ell=0}^{n-1} \frac{\tilde{q}_\ell - q_{\min}}{q_{\max} - q_{\min}} \times (S_{t_\ell} - K) \right) \\ &\leq \operatorname{ess\,sup}_{(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{Q_{\min}, Q_{\max}}} \mathbb{E} \left(\sum_{\ell=0}^{n-1} \frac{q_\ell - q_{\min}}{q_{\max} - q_{\min}} \times (S_{t_\ell} - K) \right). \end{aligned}$$

Taking the supremum, we get,

$$\begin{aligned} &\operatorname{ess\,sup}_{(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{\tilde{Q}_{\min}, \tilde{Q}_{\max}}} \mathbb{E} \left(\sum_{\ell=0}^{n-1} q_\ell \times (S_{t_\ell} - K) \right) \\ &\leq \operatorname{ess\,sup}_{(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{Q_{\min}, Q_{\max}}} \mathbb{E} \left(\sum_{\ell=0}^{n-1} \frac{q_\ell - q_{\min}}{q_{\max} - q_{\min}} \times (S_{t_\ell} - K) \right) \end{aligned}$$

Therefore,

$$\begin{aligned} P_0 &= q_{\min} \times \mathbb{E} \left(\sum_{\ell=0}^{n-1} (S_{t_\ell} - K) \right) + (q_{\max} - q_{\min}) \\ &\quad \times \operatorname{ess\,sup}_{(q_\ell)_{0 \leq \ell \leq n-1} \in \mathcal{A}_{0,0}^{\tilde{Q}_{\min}, \tilde{Q}_{\max}}} \mathbb{E} \left(\sum_{\ell=0}^{n-1} q_\ell \times (S_{t_\ell} - K) \right). \end{aligned}$$

Appendix B. On the rate of convergence. This section aims to estimate, at least numerically, the rate of convergence. We denote by U_n the price obtained after the n^{th} iteration in the training step. We make the assumption that for some constant $C > 0$

$$U_n = U_\infty + \frac{C}{n^\alpha},$$

where U_∞ is the limit of the stochastic procedure that we assume to exist. Thus,

$$\log(|U_{2n} - U_n|) = K_\alpha + \alpha \log\left(\frac{1}{n}\right), \quad \text{where } K_\alpha = \log(|C(2^{-\alpha} - 1)|).$$

Therefore the coefficient α (representing the rate of convergence following the assumption) appears to be the slope in the log – log regression of $|U_{2n} - U_n|$ against $\frac{1}{n}$.

Figures 6 and 7 suggest that both parameterizations give a rate of convergence of order $\mathcal{O}(\frac{1}{N})$ (where N is the number of iterations). This convergence rate is much faster than Monte Carlo, and suggests that our parameterizations are efficient alternatives to Longstaff-Schwartz method for this problem.

Appendix C. Estimator variance and computation time. In this section we illustrate the high variance phenomenon which may appear when pricing swing options. To this end, we compute 100 realizations of swing contract price within *case 1* setting for the three methods: *PV strat*, *NN strat* and Longstaff-Schwartz. The distributions of prices are represented in Figures 8, 9, 10.

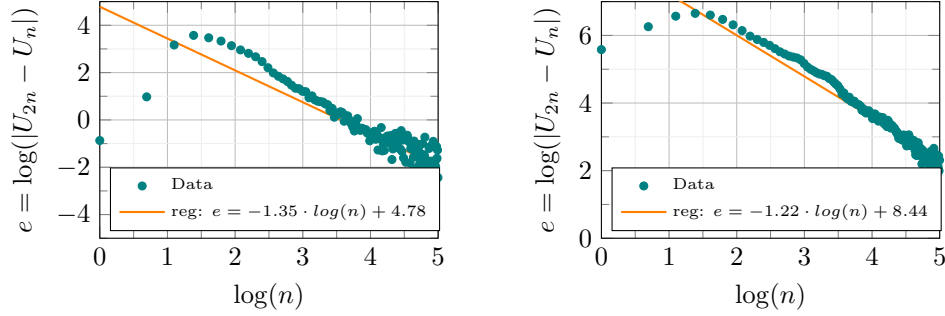


FIGURE 6. Numerical Convergence (logarithmic scale) for *PV strat.* Case 1 (left), case 2 (right)

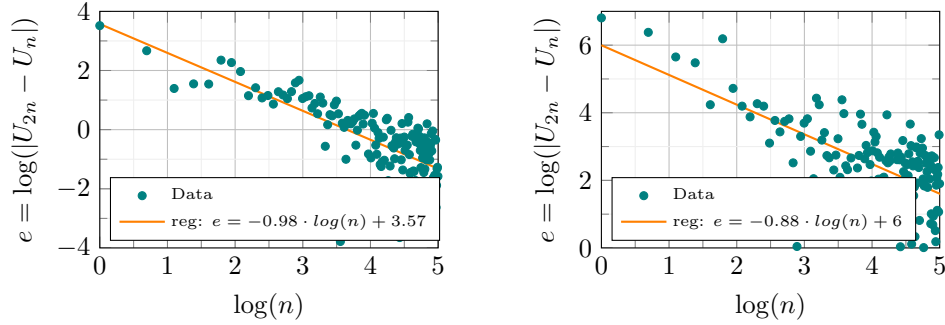


FIGURE 7. Numerical Convergence (logarithmic scale) *NN strat.* Case 1 (left), case 2 (right)

Figures 8, 9, and 10 demonstrate that prices can significantly fluctuate, regardless of the pricing method. Using 1000000 simulations is insufficient to obtain a reliable price estimate. Storage limitations prevent the Longstaff-Schwartz method from exceeding this number of simulations, and this limit is even lower when the number of exercise dates is high. On the contrary, our proposed methods allow to increase the number of simulations as needed once the strategy trained. This is possible because we can evaluate our strategies on mini-batches sequentially instead of using the entire test set.

Hereafter (see Figure 11) we present CPU time for *PV strat* and *NN strat*. We use one batch with size of 2^{14} and the same PSGLD setting.

Appendix D. Summary tables: Adam versus PSGLD. Hereafter are recorded all the results obtained using Adam and PSGLD updatings with different hyperparameters combinations. Prices are computed by averaging 100 replications of prices; each obtained using a sample size of $M_e = 10^6$. We consider a swing contract with *case 1* setting. Results are recorded in tables 14, 15, 16, 17 where we denote by n_b the number of batches used.

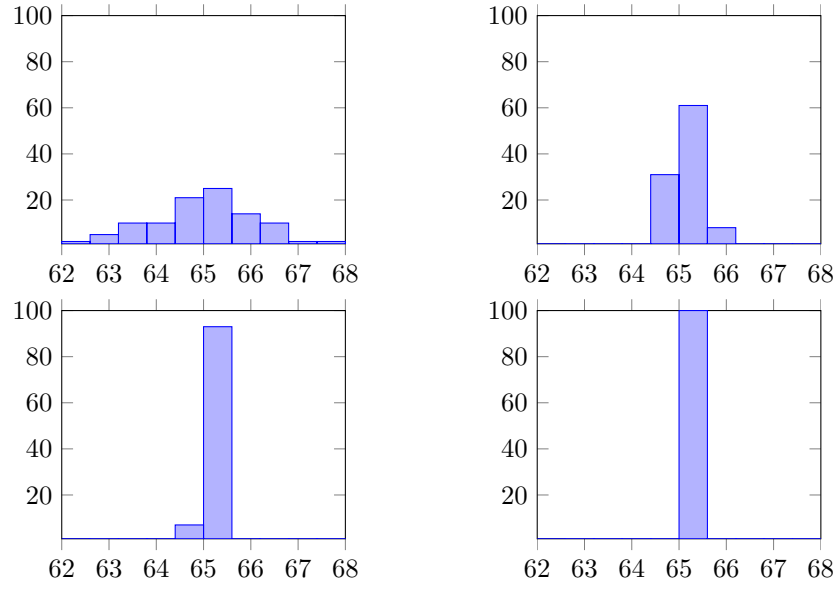


FIGURE 8. Distribution of swing prices using *PV strat.* From left to right and from top to bottom we used successively $M_e = 10^5, 10^6, 10^7, 5 \times 10^8$ simulations.

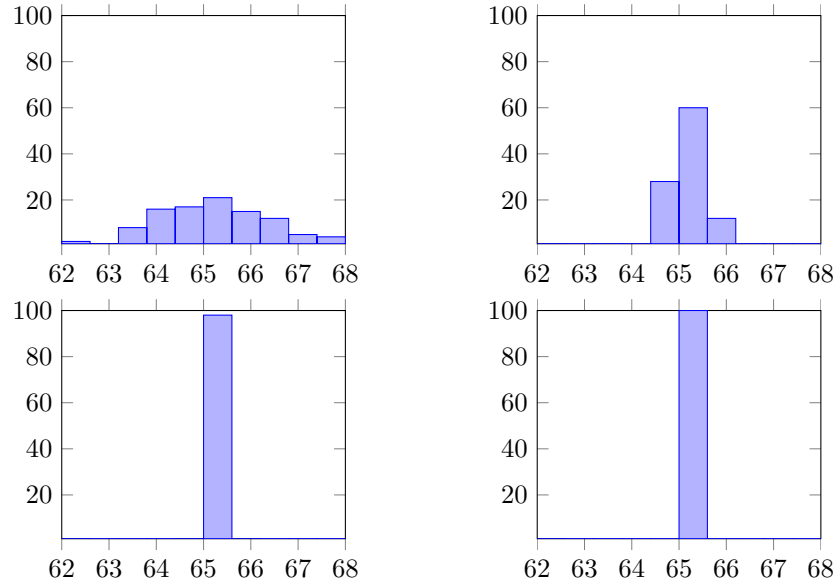


FIGURE 9. Distribution of swing prices using *NN strat.* From left to right and from top to bottom we used successively $M_{eval} = 10^5, 10^6, 10^7, 5 \times 10^8$ simulations.

Appendix E. Langevin based optimization. This appendix aims at providing some theoretical elements on Langevin dynamics as well as describing its link to

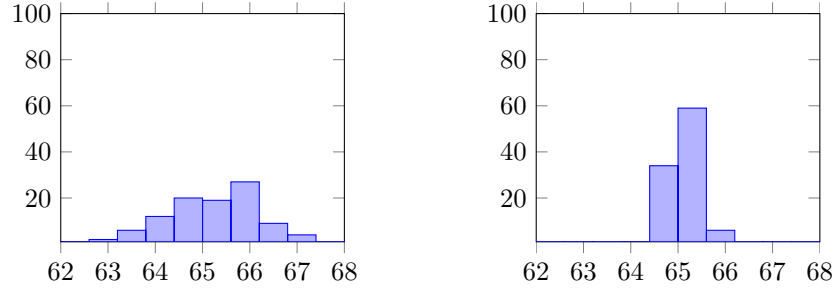


FIGURE 10. Distribution of swing prices using Longstaff-Schwartz method. From left to right we used successively $10^5, 10^6$ simulations. Higher number of simulations leads to memory overflow.

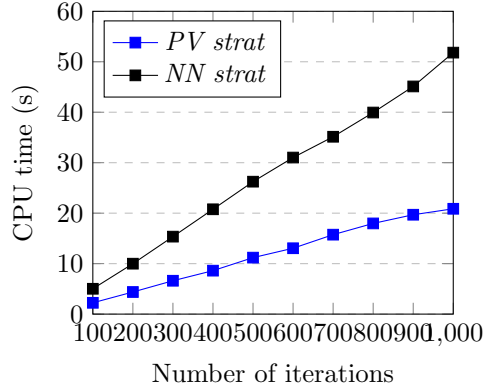


FIGURE 11. CPU time (in seconds) as a function of the number of iterations.

minimization problems. Assume that the potential J is at least C^1 with Lipschitz gradient ∇J (hence sub-quadratic) and that one can represent this gradient as the expectation of a local gradient H defined on $\Theta \times \mathbb{R}^p$

$$\nabla J(\theta) = \mathbb{E}(H(\theta, Z)), \quad (33)$$

where Z is a $\mathbb{R}^p$ -valued random vector, supposed to be simulable (it can be picked up at random a datum in a dataset). Then the resulting Stochastic Gradient Descent (SGD) reads

$$\theta_{n+1} = \theta_n - \gamma H(\theta_n, Z_{n+1}), \quad n \geq 0, \quad \theta_0 \in \Theta \quad (34)$$

which can be rewritten as

$$\theta_{n+1} = \theta_n - \gamma \nabla J(\theta_n) + \gamma \Delta M_{n+1}, \quad n \geq 0, \quad \theta_0 \in \Theta, \quad (35)$$

where $\Delta M_{n+1} = \nabla J(\theta_n) - H(\theta_n, Z_{n+1})$, $n \geq 1$, is a sequence of martingale increments (with respect to the filtration $\mathcal{F}_n = \sigma(\theta_0, Z_1, \dots, Z_n)$, $n \geq 1$).

The idea of the Langevin dynamics is to inject an *independent* d -dimensional centered (Gaussian) white noise $(W_n)_{n \geq 1}$ by considering

$$\theta_{n+1} = \theta_n - \gamma \nabla J(\theta_n) + \gamma \Delta M_{n+1} + \sqrt{2\gamma} \sigma W_{n+1}, \quad n \geq 0, \quad \theta_0 \in \Theta. \quad (36)$$

$B \times n_b$	γ	N	$\widehat{P}_0$	$\widehat{P}_0^{BG}$	Time (s)
$2^{14} \times 1$	0.1	3000	65.11 ([65.04, 65.18])	65.15 ([65.08, 65.22])	68.4
$2^{14} \times 1$	0.1	1000	65.02 ([64.95, 65.08])	65.18 ([65.12, 65.24])	21.3
$2^{12} \times 4$	0.1	3000	65.14 ([65.08, 65.21])	65.13 ([65.07, 65.20])	261.2
$2^{12} \times 4$	0.1	1000	65.15 ([65.08, 65.22])	65.17 ([65.11, 65.24])	88.8
$2^{14} \times 1$	0.01	3000	64.73 ([64.63, 64.79])	65.05 ([64.98, 65.11])	64.4
$2^{14} \times 1$	0.01	1000	63.91 ([63.83, 63.98])	64.70 ([64.62, 64.77])	22.1
$2^{12} \times 4$	0.01	3000	65.11 ([65.04, 65.18])	65.15 ([65.08, 65.22])	258.2
$2^{12} \times 4$	0.01	1000	64.81 ([64.74, 64.88])	65.03 ([64.98, 65.12])	88.3

TABLE 14. Summary table for *PV strat* using Adam. The values in brackets are confidence intervals (95%). The column “time” includes both training and valuation time.

$B \times n_b$	γ	N	$\widehat{P}_0$	$\widehat{P}_0^{BG}$	Time (s)
$2^{14} \times 1$	0.1	3000	65.22 ([65.16, 65.28])	65.23 ([65.17, 65.29])	150.4
$2^{14} \times 1$	0.1	1000	65.13 ([65.05, 65.20])	65.22 ([65.15, 65.29])	53.2
$2^{12} \times 4$	0.1	3000	65.26 ([65.21, 65.32])	65.23 ([65.17, 65.29])	603.4
$2^{12} \times 4$	0.1	1000	65.20 ([65.14, 65.26])	65.20 ([65.14, 65.26])	133.5
$2^{14} \times 1$	0.01	3000	64.78 ([62.65, 64.90])	65.06 ([64.98, 65.14])	157.1
$2^{14} \times 1$	0.01	1000	64.81 ([64.74, 64.88])	65.05 ([64.98, 65.12])	55.1
$2^{12} \times 4$	0.01	3000	65.20 ([65.15, 65.26])	65.20 ([65.14, 65.26])	607.3
$2^{12} \times 4$	0.01	1000	65.02 ([64.92, 65.11])	65.17 ([65.09, 65.24])	135.4

TABLE 15. Summary table for *NN strat* using Adam. We used a neural network architecture as follows: 2 hidden layers ($I = 2$) and 10 units per layer ($q_1 = 10, q_2 = 10$). The values in brackets are confidence intervals (95%). The time includes the training and the valuation time.

The impact of this exogenous noise $\sqrt{\gamma}\sigma W_{n+1}$ with covariance matrix $\gamma\sigma^2 I_d$ compared to that of the endogenous noise whose variance reads $\text{Var}(\gamma\Delta M_{n+1}) = \gamma^2 \|\Delta M_{n+1}\|^2$ (under standard assumptions of Robbins-Siegmund type, the martingale increments are L^2 -bounded) is clearly prominent when $\gamma, \sigma \rightarrow 0$ as long as $\gamma = o(\sigma^2)$ since then $\gamma^2 = o(\gamma)$. One shows that (see [11]) this martingale term can be neglected, mathematically speaking, in the analysis of the procedure. So one can simply focus on the procedure

$$\theta_{n+1} = \theta_n - \gamma \nabla J(\theta_n) + \sqrt{2\gamma} \sigma W_{n+1}, \quad n \geq 0, \quad \theta_0 \in \Theta \quad (37)$$

which turns to be the Euler scheme with step γ of the so-called Langevin dynamics

$$d\theta_t = -\nabla J(\theta_t)dt + \sqrt{2\gamma} \sigma dB_t, \quad \theta_0 \in \Theta, \quad (38)$$

σ	β	$\widehat{P}_0$	$\widehat{P}_0^{BG}$	Time (s)
$1e^{-5}$	0.8	65.15 ([65.09, 65.22])	65.17 ([65.10, 65.23])	22.6
$1e^{-5}$	0.7	65.13 ([65.05, 65.21])	65.15 ([65.07, 65.22])	22.1
$1e^{-5}$	0.9	65.22 ([65.16, 65.28])	65.23 ([65.18, 65.29])	21.6
$1e^{-6}$	0.8	65.19 ([65.12, 65.26])	65.21 ([65.13, 65.28])	21.3
$1e^{-6}$	0.9	65.21 ([65.15, 65.28])	65.23 ([65.17, 65.29])	21.4

TABLE 16. Summary table for *PV strat* using PSGLD. The values in brackets are confidence intervals (95%). The column “time” includes both training and valuation time. We used a learning rate equal to 0.1, one batch of size $B = 2^{14}$, $\lambda = 1 \cdot e^{-10}$ and 1000 iterations.

σ	β	$\widehat{P}_0$	$\widehat{P}_0^{BG}$	Time (s)
$1e^{-5}$	0.8	65.18 ([65.12, 65.24])	65.16 ([65.10, 65.22])	53.6
$1e^{-5}$	0.7	65.22 ([65.15, 65.30])	65.20 ([65.13, 65.27])	52.1
$1e^{-5}$	0.9	65.23 ([65.14, 65.31])	65.22 ([65.13, 65.30])	52.3
$1e^{-6}$	0.8	65.27 ([65.20, 65.35])	65.26 ([65.18, 65.33])	51.3
$1e^{-6}$	0.9	65.23 ([65.16, 65.30])	65.21 ([65.14, 65.28])	53.4

TABLE 17. Summary table for *NN strat* using PSGLD. We used $I = 2$ layers with $q_1 = q_2 = 10$ units. The values in brackets are confidence intervals (95%). The column “time” includes both training and valuation time. We used a learning rate equal to 0.1, one batch of size $B = 2^{14}$, $\lambda = 1 \cdot e^{-10}$ and 1000 iterations.

where B is a p -dimensional Brownian motion.

It is classical background that, for $\Theta = \mathbb{R}^d$, if $\int_{\mathbb{R}^d} e^{-\frac{J(\xi)}{\sigma^2}} d\xi < +\infty$, then θ_t (in (38)) converges in distribution (and for any $\mathcal{W}_p$ -Wasserstein distance) toward its invariant distribution, the Gibbs measure $\nu^{(\sigma)} = C_{J,\sigma} e^{-\frac{J}{\sigma^2}} \cdot \lambda_d$ (where λ_d is the Lebesgue measure on $\mathbb{R}^d$). Similarly, if γ is small enough, then θ_n converges in distribution toward an invariant distribution $\nu_\gamma^{(\sigma)}$ close to $\nu^{(\sigma)}$.

Note that, if the constant step γ is replaced in (37) by a decreasing sequence $(\gamma_n)_{n \geq 1}$ such that $\Gamma_n = \gamma_1 + \dots + \gamma_n \rightarrow +\infty$ and $\sum_{n \geq 1} \frac{\gamma_n}{\Gamma_n^2} < +\infty$, then θ_n converges in distribution toward $\nu^{(\sigma)}$ (see [8], see also [30]).

The use of such Langevin dynamics based algorithms for optimisation is justified by the fact that recent studies (see [15, 16]) stressed the fact that sampling from a distribution concentrated around the global minimum or global minima of J is a similar task as minimising J via certain optimisation algorithms. This is theoretically justified by the fact that $\nu^{(\sigma)}$ weakly converges to the set $\mathcal{P}(\arg \min J)$ of $\arg \min J$ -supported distribution as $\sigma \rightarrow 0$ and if $\arg \min J = \{\theta^*\}$, then $\nu^{(\sigma)} \xrightarrow{w} \delta_{\theta^*}$.

More general results, when J is C^2 and precise results about the rate of this convergence even in the case where the Hessian of J is degenerated on $\arg \min J$ can be found in the recent paper [9].

To still improve the convergence and in particular to help the (SGLD) procedure escape from local minima, practitioners introduced some *preconditioners* (see [34]) by making σ depend on θ_t in (38) (usually through a function of $\nabla J(\theta_t)$ or $J(\theta_t)$ itself). A theoretical background has been provided in [40] to explain this heuristics. First, the authors noted that the diffusion

$$d\theta_t = b(\theta_t)dt + \sqrt{2\sigma} \vartheta(\theta_t)dB_t, \quad \theta_0 \in \mathbb{R}^d \quad (39)$$

where

$$b := -\left((\vartheta\vartheta^\top) \nabla J - \sigma^2 \left[\sum_{j=1}^d \partial_{\theta_j} (\vartheta\vartheta^\top)_{ij} \right]_{i=1:d} \right) \quad (40)$$

also has $\nu^{(\sigma)}$ as a unique invariant distribution (under an ellipticity assumption on the preconditioner ϑ). The implementable version of (37), known as PSGLD, is simply the standard Euler scheme of this SDE with step $\gamma > 0$

$$\theta_{n+1} = \theta_n - \gamma b(\theta_n) + \sqrt{2\gamma} \sigma \vartheta(\theta_n) W_{n+1}, \quad n \geq 0, \quad y_0 \in \mathbb{R}^d. \quad (41)$$

(In fact the version investigated in [40] has a decreasing step $(\gamma_n)_{n \geq 1}$ and directly converges toward $\nu^{(\sigma)}$).

Practitioners usually consider diagonal preconditioners of the form

$$\forall \theta = (\theta^1, \dots, \theta^d) \in \mathbb{R}^d, \quad \vartheta\vartheta^\top(\theta) = \text{Diag}\left((\varphi(\partial_{\theta^1} J(\theta)))^2, \dots, (\varphi(\partial_{\theta^d} J(\theta)))^2 \right).$$

Numerical experiments carried out by practitioners suggest that the resulting correcting term in the drift, which is computationally demanding in terms of complexity, can be neglected in (40) without damage for practical implementation (see [34]).

A simulated annealing version of the procedure (37) has been introduced and analysed in [11] in which γ and σ are no longer constant but of the form $\gamma_n \downarrow 0$, $\Gamma_n \rightarrow +\infty$ and $\sigma_n = A/\sqrt{\log(n+1)}$. In this setting, it is proven that $\theta_n \rightarrow \arg \min J$ in probability.

REFERENCES

- [1] J.-G. Attali and G. Pagès, Approximations of functions by a multilayer perceptron: A new approach, *Neural Networks*, **10** (1997), 1069-1081.
- [2] A. Bachouch, C. Huré, N. Langrené and H. Pham, [Deep neural networks algorithms for stochastic control problems on finite horizon: Numerical applications](#), *Methodology and Computing in Applied Probability*, **24** (2021), 143-178.
- [3] O. Bardou, S. Bouthemy and G. Pagès, [Optimal quantization for the pricing of swing options](#), *Applied Mathematical Finance*, **16** (2009), 183-217.
- [4] O. Bardou, S. Bouthemy and G. Pagès, [When are swing options bang-bang?](#), *International Journal of Theoretical and Applied Finance*, **13** (2010), 867-899.
- [5] C. Barrera-Esteve, F. Bergeret, C. Dossal, E. Gobet, A. Meziou, R. Munos and D. Reboul-Salze [Numerical methods for the pricing of swing options: A stochastic control approach](#), *Methodology and Computing in Applied Probability*, **8** (2006), 517-540.
- [6] A. R. Barron, [Universal approximation bounds for superpositions of a sigmoidal function](#), *IEEE Transactions on Information Theory*, **39** (1993), 930-945.
- [7] A. R. Barron, Approximation and estimation bounds for artificial neural networks, *Machine Learning*, **14** (1994), 115-133.
- [8] G. K. Basak and R. N. Bhattacharya, Stability in distribution for a class of singular diffusions, *Annals of Probability*, **20** (1992), 312-321.

- [9] P. Bras, [Convergence rates of Gibbs measures with degenerate minimum](#), *Bernoulli*, **28** (2022), 2431-2458.
- [10] P. Bras, [Langevin algorithms for very deep neural networks with application to image classification](#), *Procedia Computer Science*, **222** (2023), 303-310.
- [11] P. Bras and G. Pagès, [Convergence of Langevin-simulated annealing algorithms with multiplicative noise](#), *Mathematics of Computation*, **93** (2024), 1761-1803.
- [12] P. Bras and G. Pagès, Langevin algorithms for markovian neural networks and deep stochastic control, In *2023 International Joint Conference on Neural Networks (IJCNN)*, **48** (2023), 1-8.
- [13] M. Broadie and P. Glasserman, A stochastic mesh method for pricing high-dimensional american options, *Journal of Computational Finance*, **7** (2004), 35-72.
- [14] G. Cybenko, [Approximation by superpositions of a sigmoidal function](#), *Mathematics of Control, Signals and Systems*, **2** (1989), 303-314.
- [15] A. S. Dalalyan, [Theoretical guarantees for approximate sampling from smooth and log-concave densities](#), *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, **79** (2017), 651-676.
- [16] A. S. Dalalyan, Further and stronger analogy between sampling and optimization: Langevin monte carlo and gradient descent, In *Annual Conference Computational Learning Theory*, 2017.
- [17] A. Défossez, L. Bottou, F. Bach and N. Usunier, A simple convergence proof of adam and adagrad, *Transactions on Machine Learning Research*, 2022.
- [18] R. A. DeVore, B. Hanin and G. Petrova, [Neural network approximation](#), *Acta Numerica*, **30** (2020), 327-444.
- [19] A. Durmus and É. Moulines, [Nonasymptotic convergence analysis for the unadjusted Langevin algorithm](#), *The Annals of Applied Probability*, **27** (2017), 1551-1587.
- [20] A. Durmus and É. Moulines, [High-dimensional bayesian inference via the unadjusted langevin algorithm](#), *Bernoulli*, **25** (2019), 2854-2882.
- [21] B. Fehrman, B. Gess and A. Jentzen, Convergence rates for the stochastic gradient descent method for non-convex objective functions, *Journal of Machine Learning Research*, **21** (2020), 1-48.
- [22] S. B. Gelfand and S. K. Mitter, [Recursive stochastic algorithms for global optimization in \$\mathbf{R}^d\$](#) , In *SIAM J. Control Optim.*, **29** (1991), 999-1018.
- [23] E. Gobet and R. Munos, [Sensitivity analysis using itô-malliavin calculus and martingales, and application to stochastic optimal control](#), *SIAM J. Control and Optimization*, **43** (2005), 1676-1713.
- [24] J. Han and E. Weinan, Deep learning approximation for stochastic control problems, *ArXiv*, abs/1611.07422, 2016.
- [25] K. Hornik, M. B. Stinchcombe and H. L. White, Multilayer feedforward networks are universal approximators, *Neural Networks*, **2** (1989), 359-366.
- [26] P. Jaillet, D. Lamberton and B. Lapeyre, [Variational inequalities and the pricing of american options](#), *Acta Applicandae Mathematica*, **21** (1990), 263-289.
- [27] P. Jaillet, E. I. Ronn and S. Tompaidis, Valuation of commodity-based swing options, *Manag. Sci.*, **50** (2004), 909-921.
- [28] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.
- [29] H. J. Kushner and G. Yin, *Stochastic Approximation and Recursive Algorithms and Applications*. Springer New York, NY, 2003.
- [30] D. Lamberton and G. Pagès, [Recursive computation of the invariant distribution of a diffusion](#), *Bernoulli*, **8** (2002), 367-405.
- [31] B. Lapeyre and J. Lelong, [Neural network regression for bermudan option pricing](#), *Monte Carlo Methods and Applications*, **27** (2021), 227-247.
- [32] A. Lari-lavassani and M. Simchi, A discrete valuation of swing options, *Canadian Applied Mathematics Quarterly*, **9** (2001), 35-73.
- [33] Y. Lei, T. Hu, G. Li and K. Tang, [Stochastic gradient descent for nonconvex learning without bounded gradient assumptions](#), *IEEE Transactions on Neural Networks and Learning Systems*, **31** (2019), 4394-4400.
- [34] C. Li, C. Chen, D. E. Carlson and L. Carin, Preconditioned stochastic gradient langevin dynamics for deep neural networks, In *AAAI Conference on Artificial Intelligence*, 2015.

- [35] F. A. Longstaff and E. S. Schwartz, Valuing american options by simulation: A simple least-squares approach, *The Finance*, 2001.
- [36] S. Massaroli, M. Poli, S. Peluchetti, J. Park, A. Yamashita and H. Asama, [Learning stochastic optimal policies via gradient descent](#), *IEEE Control Systems Letters*, **6** (2022), 1094-1099.
- [37] E. Moulines and F. Bach, Non-asymptotic analysis of stochastic approximation algorithms for machine learning. In *Advances in Neural Information Processing Systems*, Curran Associates, Inc., **24** (2011).
- [38] A. Neelakantan, L. Vilnis, Q. V. Le, I. Sutskever, L. Kaiser, K. Kurach and J. Martens, Adding gradient noise improves learning for very deep networks, *ArXiv*, abs/1511.06807, 2015.
- [39] A. Nemirovski and D. B. Yudin, *Problem Complexity and Method Efficiency in Optimization*, John Wiley & Sons Ltd, 1983.
- [40] G. Pagès and F. Panloup, [Unadjusted Langevin algorithm with multiplicative noise: Total variation and Wasserstein bounds](#), *Annals of Applied Probability*, **33** (2023), 726-779.
- [41] G. Pagès, H. Pham and J. Printems, *Optimal Quantization Methods and Applications to Numerical Problems in Finance*, Birkhäuser Boston, Boston, MA, 2004.
- [42] S. J. Pan and Q. Yang, A survey on transfer learning, *IEEE Transactions on Knowledge and Data Engineering*, **22** (2010), 1345-1359.
- [43] N. Qian, On the momentum term in gradient descent learning algorithms, *Neural Networks : The Official Journal of the International Neural Network Society*, **12** (1999), 145-151.
- [44] H. E. Robbins, [A stochastic approximation method](#), *Annals of Mathematical Statistics*, **22** (2007), 400-407.
- [45] L. C. G. Rogers, [Monte carlo valuation of american options](#), *Mathematical Finance*, **12** (2002), 271-286.
- [46] O. Shamir and T. Zhang, Stochastic gradient descent for non-smooth optimization: Convergence results and optimal averaging schemes, *ArXiv*, abs/1212.1824, 2012.
- [47] A. C. Thompson, Valuation of path-dependent contingent claims with multiple exercise decisions over time: The case of take-or-pay, *Journal of Financial and Quantitative Analysis*, **30** (1995), 271-293.
- [48] M. Welling and Y. W. Teh, Bayesian learning via stochastic gradient langevin dynamics, In *International Conference on Machine Learning*, 2011.
- [49] M. A. Zinkevich, Online convex programming and generalized infinitesimal gradient ascent, In *International Conference on Machine Learning*, 2003.
- [50] F. Zou, L. Shen, Z. Jie, W. Zhang and W. Liu, A sufficient condition for convergences of adam and rmsprop, *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, (2018), 11119-11127.

Received December 2023; 1st revision February 2024; 2nd revision March 2024; early access April 2024.